

学校代号 10532

学 号 S1710W0764

分 类 号 TP391

密 级



湖南大学
HUNAN UNIVERSITY

工程硕士学位论文

学术论文 PDF 结构解析技术的研究

学位申请人姓名 周忆莲

培 养 单 位 信息科学与工程学院

导师姓名及职称 何黎刚 教授 涂旭平 高工

学 科 专 业 计算机技术

研 究 方 向 文本挖掘

论文提交日期 2020 年 5 月 10 日

学校代号:10532

学 号:S1710W0764

密 级:

湖南大学硕士学位论文

学术论文 PDF 结构解析技术的研究

学位申请人姓名: 周忆莲

导师姓名及职称: 何黎刚 教授 涂旭平 高工

培 养 单 位: 信息科学与工程学院

专 业 名 称: 计算机技术

论文提交日期: 2020 年 5 月 10 日

论文答辩日期: 2020 年 5 月 29 日

答辩委员会主席: 陈浩 教授

Research on PDF Structure Analysis Technology of Academic Papers

By

ZHOU Yilian

B.E.(Hunan Normal University) 2017

A thesis submitted in partial satisfaction of the

Requirements for the degree of

Master of Engineering

in

Computer Technology

in the

Graduate School

of

Hunan University

Supervisor

Professor He Ligang

Senior Engineer Tu Xuping

May, 2020

湖南大学

学位论文原创性声明

本人郑重声明:所呈交的论文是本人在导师的指导下独立进行研究所取得的科研成果。除了文中特别加以标注引用的内容外,本论文不包含任何其他个人或集体已经发表或撰写的成果作品。对本文的研究做出重要贡献的个人和集体,均已在文中以明确方式标明。本人完全意识到本声明的法律后果由本人承担。

作者签名:

日期: 年 月 日

学位论文版权使用授权书

本学位论文作者完全了解学校有关保留、使用学位论文的规定,同意学校保留并向国家有关部门或机构送交论文的复印件和电子版,允许论文被查阅和借阅。本人授权湖南大学可以将本学位论文的全部或部分内容编入有关数据库进行检索,可以采用影印、缩印或扫描等复制手段保存和汇编本学位论文。

本学位论文属于

1、保密□,在_____年解密后适用本授权书。

2、不保密□。

(请在以上相应方框内打“√”)

作者签名:

日期: 年 月 日

导师签名:

日期: 年 月 日

摘 要

PDF 为数字化信息传播及电子文档发行提供了很大便利,它也成为了目前学术论文的主要载体。这些海量的学术论文资源经过加工、整合、重编可以形成可重用的论文结构对象,这些结构对象将作为学术论文应用的输入数据。在理想情况下,PDF 的元数据是可以存储结构化的信息,然而 PDF 是一种基于布局的格式,它并不提供结构信息,因此,对 PDF 学术论文结构内容的提取是学术资源挖掘的主要挑战。

本文从文本元素和矢量元素两方面介绍了解析的关键问题,对影响其解析结果的因素进行了分析并提出了解决方案,针对文本区域提出了基于文本坐标规律的识别方法,将文本块内的文本行的坐标规律作为文本块扩充的条件;针对非文本区域提出基于最大矩形区域扩充的识别方法,将正文看成以页面分割的各个区域,对提取的矢量元素按左右下方向扩充,以最大范围地包含矢量元素及非正文文本,识别出非文本块。考虑到识别出的块与块之间的冗余与重叠,设计了块的合并算法,并对于多栏版面的 PDF 渲染顺序不正确引起的生成的块顺序不正确的问题设计了排序算法。

实验结果表明,本文中提出的识别算法能很好地提取 PDF 结构信息,能够自动化地对学术资源进行抽取与处理,有利于 PDF 在学术论文领域中的进一步利用,对当前学术论文资源知识挖掘研究具有重要意义。

关键字:PDF 解析; 文本; 矢量图像

Abstract

PDF provides great convenience for digital information dissemination and electronic document distribution, and it has become the main carrier of current academic papers. These large amounts of academic paper resources can be processed, integrated, and reorganized to form reusable paper structure objects. These structure objects will be used as input data for academic paper applications. In an ideal situation, the metadata of PDF can store structured information. However, PDF is a layout-based format, which does not provide structural information. Therefore, the extraction of the structure of PDF academic papers is the main challenge for academic resource mining.

This paper introduces the key issues of analysis from the two aspects of text elements and vector elements, analyzes the factors that affect the analysis results, and proposes solutions. For text areas, a recognition method based on the rules of text coordinates is proposed, and for non-text areas, a recognition method based on the expansion of the largest rectangular area is proposed. Considering the redundancy and overlap between the identified blocks, a block merging algorithm was designed, and a sorting algorithm was designed for the problem of incorrect block order caused by the incorrect PDF rendering order of the multi-column layout.

The experimental results show that the recognition algorithms proposed in this paper can extract PDF structural information well and can automatically extract and process academic resources, and it is conducive to the further use of PDF in the field of academic papers and researches on the current academic paper resource knowledge of great significance.

Key Words: PDF document analysis; text; vector image

目 录

学位论文原创性声明和学位论文版权使用授权书	I
摘 要	II
Abstract	III
目 录	IV
插图索引	VI
附表索引	VIII
第 1 章 绪 论	1
1.1 课题背景及意义	1
1.2 国内外研究现状	2
1.2.1 基于学习方法研究现状	2
1.2.2 基于规则方法研究现状	3
1.2.3 其它研究方法	4
1.2.4 现有解析工具	5
1.3 论文主要工作	6
1.4 论文内容安排	6
第 2 章 研究相关技术	8
2.1 PDF 文档对象	8
2.2 PDF 文档结构	9
2.2.1 PDF 物理结构	9
2.2.2 PDF 逻辑结构	11
2.3 PDF 图形元素	12
2.3.1 PDF 图形内容流	12
2.3.2 PDF 光栅图像	14
2.3.3 PDF 矢量图像	15
2.4 PDF 文本元素	15
2.4.1 PDF 字体对象	15
2.4.2 PDF 文本位置	16
2.4.3 PDF 文本显示	16
2.5 PDFTron 框架	17
2.6 本章小结	19
第 3 章 文本区域识别方法	20

3.1 关键因素及影响分析	20
3.1.1 文本解析关键因素	20
3.1.2 文本解析影响分析	21
3.2 基于文本坐标规律的识别方法	23
3.2.1 全局信息获取	24
3.2.2 文本行坐标获取	25
3.2.3 文本块生成	27
3.3 本章小结	29
第 4 章 非文本区域识别方法	30
4.1 关键因素及影响分析	30
4.1.1 图形解析关键因素	30
4.1.2 图形解析影响分析	30
4.2 基于最大矩形区域扩充的识别方法	31
4.2.1 PATH 路径重绘	32
4.2.2 PATH 路径还原	34
4.2.3 矢量图形生成	36
4.3 本章小结	37
第 5 章 块的合并与排序	38
5.1 块的合并算法	38
5.2 块的排序算法	39
5.3 本章小结	40
第 6 章 实验评估	41
6.1 实验设计与过程	41
6.2 实验结果	45
6.2.1 文本	46
6.2.2 图像	49
6.2.3 表格	52
6.3 本章小结	55
结论	56
参考文献	58
附录 A 攻读硕士学位期间所发表的学术论文	63
附录 B 攻读硕士学位期间参加科研项目及申请的专利	64
致 谢	65

插图索引

图 2.1	更具可读性的字典对象	9
图 2.2	流对象的示例	9
图 2.3	PDF 物理结构	10
图 2.4	对象的定义	10
图 2.5	PDF 逻辑结构	11
图 2.6	Trailer dictionary 举例	11
图 2.7	PATH 绘制命令	13
图 2.8	光栅图像在资源字典中的定义	14
图 2.9	文本对象在流中的定义	17
图 3.1	文本行的渲染顺序有误	22
图 3.2	文本区域识别流程图	23
图 3.3	数据结构图	24
图 3.4	全局信息类	24
图 3.5	三种类型的文本	25
图 3.6	PDFTron 提取行效果	26
图 3.7	PDFTron 提取行高	27
图 3.8	文本行 y1 与 y2 坐标值	27
图 3.9	文本块生成算法	28
图 4.1	非文本区域识别流程图	32
图 4.2	重绘后的 PATH	33
图 4.3	PDF 坐标系	34
图 4.4	Matrix2D 矩阵	35
图 4.5	还原后的 PATH	35
图 4.6	矢量图形生成块生成算法	36
图 4.7	表格提取	37
图 5.1	文本块存在交叉	38
图 5.2	PATH 存在交叉	38
图 5.3	块的合并算法	39
图 5.4	块的排序算法	40
图 6.1	PDF 学术论文上传	41
图 6.2	PDF 学术论文管理	42

图 6.3	PDF 学术论文自动化生成块信息	42
图 6.4	自动生成基础上人工调节块信息	43
图 6.5	JSON 结构图示	44
图 6.6	论文页面识别效果	45
图 6.7	PDFBox 提取文本	46
图 6.8	文本行提取	47
图 6.9	文本块坐标合并	48
图 6.10	文本块识别对比直方图	49
图 6.11	原始图像	50
图 6.12	PDFBox 提取的图像	50
图 6.13	矢量图像识别	51
图 6.14	矢量图像最终坐标	51
图 6.15	矢量图形块识别对比直方图	52
图 6.16	表格识别	53
图 6.17	文献 53 中表格延伸示意图	54
图 6.18	表格块识别对比直方图	54

附表索引

表 2.1	文件信息字典条目	12
表 2.2	图形运算符	13
表 2.3	文本状态参数及运算符	16
表 2.4	文字定位运算符	16
表 6.1	对比算法中文本块识别统计	46
表 6.2	本算法中文本块识别统计	48
表 6.3	对比算法中矢量图形块识别统计	50
表 6.4	本算法中矢量图像块识别统计	52
表 6.5	本算法中表格识别统计	53
表 6.6	对比算法中表格识别统计	53

第 1 章 绪 论

1.1 课题背景及意义

PDF(Portable Document Format, 便携文档格式)是 Adobe 公司创建的用于进行全球电子文档存储与分发的一种电子文件格式^[1]。PDF 基于 PostScript 页面描述语言, PostScript 是解释器中运行的关于页面描述语言的子集, 它主要用于生成页面布局与图像^[2]。但 PostScript 并不便于对当今的计算机在屏幕上的使用, 特别是在随机访问文档某一页的场景下, 因此 PDF 逐步取代了 PostScript^[3]。目前, PDF 已由国际标准化组织(ISO)和超过 75 个国家/地区进行了标准化, 能够很好地代替纸质传媒来进行文档的传播^[4]。在当今的工作与学习环境中, PDF 的采用率很高, 它作为传统纸张的替代品已经无处不在, 并且已经拥有了各种领域重要的业务数据, 凭借优越的特性 PDF 已经成为电子文档的主流格式, 被广泛应用于互联网中电子文档的存储与分发, 成为数字化信息传播最理想的介质。PDF 所具有的特性有以下几方面:

(1)独立于平台: PDF 是一种可移植的格式, 它与查看此文档的软件、硬件以及操作系统平台独立, 它被设计为一种输出文档, 用户能够捕获 PDF 文档的原始布局, PDF 文档在各个系统中是通用的, 它能够支持用户在任何设备上使用相同的布局方式来查看和打印该文档。

(2)不易更改: 与他人共享文件或照片时, 由于 PDF 无法编辑, 使得对方无法轻松更改, 来回浏览文件很方便。并且可以通过选项设置使用密码保护和访问权限, 保护 PDF 文档内容或整个文件, 因此我们可以看到带水印、密码保护和数字签名的文档。

(3)内容丰富, PDF 文档可以集成各种类型的内容, 所有图像、文本、矢量图形、动画、音频文件、按钮这些元素都可以合并组合成文件, 最值得一提的是, PDF 格式具有动画的交互性的全部功能, 它与印刷版不同, 可以包含特殊的链接和动画, 链接可以增加交互性集成, 扩展了对于该主题的学习, 而不必在实际的文档中嵌入过多的信息, 其中, 外部文件和有关网站都可以以链接形式穿插在 PDF 文档中, 而动画则可以更好地对文档中具体给定数据作出说明或展示。

然而, 尽管 PDF 有许多优秀特性, 但还是存在局限性, 它最初的目的是保留文档的布局和保护文档的内容, 使得在任何平台上都可以一致的显示, 因此, PDF 是作为文档的交换格式而被开发, 这也解释了 PDF 难以被再次编辑的原因, 并且对于提取 PDF 文档结构信息或将完整的文档转换为其它可编辑格式也是一种挑战。对于学术论文而言, 目前学术论文 PDF 文档的数量在以极其迅速的速度增加,

在这海量的文档内含有大量的语义信息，但 PDF 文档的核心是指令流，它们决定了元素在 PDF 页面上的绘制方式，其中，文本数据不包含语义信息，它被存储在 PDF 中时大部分的语义内容就已经丢失，所有原先隐含在其中的结构信息都被存储为页面上的字符，即它并不会存储为单词或者段落，而是会作为在页面指定位置上所绘制的字符来存储，因此这些语义结构信息不能被简单提取。由于学术论文中包含大量语义结构信息，它们将作为学术论文相关应用的输入数据，如果我们能得到这些结构信息，就可以针对不同类型的元素执行不同的操作，在其它领域的 PDF 的应用中一样如此，因此对 PDF 结构信息提取的研究在学术圈是非常有意义的。本实验提出的解决方案主要针对学术论文中结构信息的提取，同样适用于其它领域 PDF。

目前已经有了一些工具可以实现对 PDF 及学术论文结构信息的提取，但是现有的这些工具往往只适用于一般的情况，很难根据系统特定的需求进行动态调整，或完全达到用户所希望的功能。在使用 PDF 时，并非所有类型的 PDF 都一样，有些 PDF 只包含表格，有些 PDF 版面只包含一栏，而有些包含两栏，有些 PDF 包含页眉，因此不同类型的 PDF 需要不同的处理方式，对于 PDF 解析相关的研究因运而生，PDF 的解析工作待提取的信息主要为两类：文本元素和矢量元素。本实验提出了一种细粒度地对学术论文 PDF 中的文字与矢量元素进行解析的方法，该方法可以直接从 PDF 内容流中提取文字和矢量元素并根据特征将其表示为具体的语义结构的方法，这种级别适合于对学术论文文档的结构理解，同样也适用于其它领域的 PDF 文档。学术论文 PDF 中的结构信息是相关大数据、深度学习、数据挖掘应用的数据来源，而目前 PDF 学术论文结构信息需要人工进行提取，无法应对当前海量的学术论文 PDF 大数据，因此自动化地对学术资源进行抽取与处理是未来的大趋势，本课题具有重要的研究意义以及良好的应用前景。

1.2 国内外研究现状

目前，国内外对 PDF 文档解析的研究工作取得了一定的进展，根据所基于的技术可以将 PDF 文档解析方法分成两类：1.基于学习方法，2.基于规则方法，这两类方法所提出的算法在某个领域或 PDF 中的某种文档结构方面都能很好地得到应用。同时，也涌现了各种 PDF 解析的工具及库，给开发者封装了底层解析的接口，使得开发者可以借助其开发自己的应用满足特定的需求。

1.2.1 基于学习方法研究现状

在过去的几年里，基于学习的方法是分析 PDF 的主流方法，这些研究专注于 PDF 中的某一具体结构信息，Z.Wang 为实现从 PDF 文档中提取数学表达式，提出了一种基于多阶段的无监督建模方法，从基于 PDF 对象元数据的符号级别分析开始，随后的两个阶段利用语义空间试探法将多个 ME 符号合并为内联 ME 和显

示 ME, 该算法平均只需要 1.09s 的时间即可处理一页, 比其他现有方法要快^[5]。C.Tensmeyer 提出了一种表结构的深度分割和合并方法, 采用两个神经网络深度学习模型(分割与合并)来提取 PDF 中的表格, 通过预测应该合并哪些网格元素以恢复跨越多行和多列的单元格来预测基本的网格模式^[6]。从学术图形中提取场景文本尚未得到解决, 难点在于难以找到不规则放置的文本行, D.Morris 提出的方法放弃了预处理, 转而使用深度卷积神经网络进行文本行定位, 该方法的网络体系结构非常适合从学术图形中提取文本^[7]。Anukriti Bansal 提出了一种新颖的基于学习的表格识别框架来从扫描的文档图片中识别表格, 该方法被设计为一个结构化的标记问题, 它学习文档的布局并将其各种实体标记为表标头、表尾、表单元格和非表区域^[8]。L.Gao 将卷积神经网络和递归神经网络模型结合并细化, 根据特征和视觉特征检测和提取公式, 在算法中通过一系列策略来训练和优化深层网络, 例如隐式类下采样策略, 可以减少公式与其他页面元素之间的不平衡, 通过组合自下而上和自上而下的布局分析, 还重新设计了区域提议方法, 以生成适度的公式候选, 大大提高了检测方法的鲁棒性^[9]。本文所提出的学术论文 PDF 结构解析技术的研究, 可以对 PDF 中的文本块、图像、以及表格进行提取, 能满足学术论文资源的整合及应用。

1.2.2 基于规则方法研究现状

但是, 在一些研究中也证明, 简单的启发式在具体的问题上其表现优于机器学习方法。任林涛针对科技论文结构进行了有效信息的提取, 根据其格式与视觉上的特征, 提取出科技论文的标题、作者、摘要以及论文关键字, 作为后续论文分类应用的输入数据^[10]。Boukhers 提出一个端到端的方法来提取和分割参考文献, 学习了相关方案中使用的引用的不同特征, 通过遵循概率的方法来减少错误累积^[11]。PDF 文档中的数学对象(MO)对于理解已出版的科学、技术、工程文档中的本体和数学本质至关重要, Donald Beyette 提出了一个半自动标注 MO 算法, 它使用 PDF 文档及其相应的 LaTeX 并且提出了一种最短的独特字符串匹配技术, 使 PDF 页面与 LaTeX 文件对齐^[12]。人们越来越关注挖掘已文档中的图形并将其各自的标题用途知识来源, 可以自动为图像生成字幕的图像字幕系统引起了很多兴趣, 而从 PDF 中获取图形和标题的第一步仍然存在一些问题, 不同于现有的通过处理 PDF 文档的原始编码内容来提取图形的方法, Pengyuan Li 将文本与图形内容分开, 利用布局信息来检测和消除图形和标题的歧义, 从 PDF 中提取数字和标题及图片, 生成图像字幕^[13]。自动提取可靠的财务数据是一项重要且极具挑战性的任务, Benoit Potvin 提出了一种利用两种简单的启发式方法提取财务信息的新方法, 该方法基于这样的思想, 即在非结构化但视觉上丰富的文档中信息的位置是语义的指标^[14]。Alexey Shigarov 讨论了一种新的无标签 PDF 文档表结构识别系统, 它被表示为一组可配置参数和用于恢复表单元格的自组织启发式, 产生标记来标识逻

辑阅读顺序、段落、图表和表格^[15]。Joeran Beel 介绍了 Docear 的 PDF 检查器 (DPI), DPI 通过应用一个简单的启发式方法从学术 PDF 文件中提取标题, 这个简单的启发式实现了大约 70% 的准确性^[16]。Chao 提取了一种基于规则地实现图表识别的方法, 使用自底向上的方法识别 PDF 文档中的图表, 通过应用规则对图表部件及图表文字进行扩充合并^[17]。在这些研究中, 算法往往针对某一具体类型的 PDF 制定规则, 而不具普适性, 本文所提出的学术论文 PDF 结构解析技术的研究, 能够对其它领域的 PDF 也进行解析, 提取出其中的文本块、图像及表格。

1.2.3 其它研究方法

有相关研究将基于学习与基于规则这两种主要方法结合在一起, 于文浩使用基于规则和基于学习的方法来自动进行表格读取, 以从纸质 PDF 表格中提取和管理实验证据^[18]。简·霍夫斯韦尔 (Jane Hoffswell) 描述了基于现有机器学习和基于规则的技术提取, 分析和解析 PDF 表^[19]。X.Wang 提出了一种基于字体设置的贝叶斯模型来提取 PDF 文件中的数学表达式, 在该算法中首先制定规则来识别公式, 然后基于公式的结果中倾向的特定样式, 使用贝叶斯来判断文档布局^[20]。

还有一些方法通过将 PDF 转换为包含 XML, HTML 等结构化信息的其他文档格式, 但是这些转换过程可能导致有价值信息的丢失。Rastan R 介绍了端对端表的提取和理解该系统采用 PDF 文件, 并自动生成一组结构化的 XML 和 CSV 文件^[21]。Perez-Arriaga 提出了一种 TAO 系统, 该系统可以自动检测和组织 PDF 文档中的表格中的信息, 它使用了基于 K 最近的处理。邻居方法和布局启发式方法^[22]。宋艳娟提出了基于 XML 的 HTML 和 PDF 信息抽取技术的研究, 利用 XSLT 和 XPath 技术在数据定位以及文档转换方面的优势来对信息进行抽取, 并且构造了基于 XML 的 PDF 信息抽取系统^[23]。对标签 PDF 文档结构作了研究, 通过 PDF 内部标签建立了文档逻辑结构与内容之间的联系机制, 并用 Plug-in 编程接口做了实现^[24]。Rahman 和 Alam 对将 PDF 文档转换为 HTML 文档的过程进行了详细描述, 将文档图像分析技术应用到了 PDF 文档的逻辑及版面分析过程中^[25]。

基于内容流的提取也有不错的表现, 张秀秀给出了 PDF 文件的解析流程并从解析后的内容流中提取文本内容, 为数字知识库建设研发元数据自动抽取工具^[26]。陈云榕等人通过搜寻 PDF 页面树, 针对采用 DCTDecoder 编码的图像数据解析, 并将其以 JPEG 文件格式存储, 实现了从 PDF 文件中提取 JPEG 图像的功能^[27]。

不仅如此, PDF 解析还用于安全领域, 亚历山大·乔丹提出通过使用抽象解释静态推理其可能的行为来检测 PDF 中的恶意代码, 实验结果表明对规避攻击更具弹性, 并且提供了可解释的报告^[28]。李敏通过静态和动态分析来检测带有 JavaScript 的恶意 PDF 文档, 并且建立了原型检测系统, 首先通过静态分析的方法对从 PDF 文档中提取的 JavaScript 代码进行模糊处理, 并且模拟动态分析期间的代码执行^[29]。H.Bast 小组也做出了很多贡献, 他们展示了如何从并行 TeX 和

PDF 数据构造中构造出一个基本上任意大小的高质量基准。arXiv.org 提供的 12098 篇科学文章的基准，并使其公开可用^[30]。

1.2.4 现有解析工具

目前，有大量从 PDF 中提取信息的工具及研究，它们都有各自的优缺点，尽管丰富多样，但还没有占据绝对优势的工具存在，这类工具通常情况下只适用于一般情况，无法完全实现用户所希望的功能和达到系统特定的需求。对于工具的选择取决于用户需求，用户可以在现有研究和工具的基础上进行扩展和改进来完善功能。现有的工具中，pdfminer3k 是 pdfminer 的 Python3 端口，用于从 PDF 中提取信息，它与其他相关工具不同的地方在于只专注于获取和分析文本数据，对于表格数据，能提取出表格中的文字，但不能完整地提取出表格，无法与正文区分^[31]。Apache PDFBox 基于 Java 的 PDF 文档处理开放库，它可对 PDF 进行创建、提取、转换等操作，提供了文本、图像操作的接口，是目前使用得较多的一个工具^[32]。牛永洁也提出了基于 PDFBox 抽取学术论文信息的实现，对 PDF 格式的学术论文按照制定规则进行信息的提取，主要包括学术论文的标题、作者、单位、关键词等信息^[33]，但只针对于文本元素，而不包含图像与表格的处理。基于 .Net 下的 iTextSharp 也是用于 PDF 处理的开源 SDK，是 .Net 语言下使用较多的开源 SDK，能够对 PDF 进行编辑、解析和移植^[34]。PoDoFo 是基于 C++ 的 PDF 操作类库，实现了对 PDF 常用的编辑工作，并提供了解析器用于从 PDF 文件中提取信息^[35]，对于其中的文本提取，目前只适用于简单的 PDF。Pdftotext 是一个 PDF 的转换工具，可以将 PDF 格式的文件转换为 txt 文件，但该工具目前还无法识别出文本中的段落边界^[36]。Pdftohtml 是另一款对 PDF 进行转换的工具，它支持将 PDF 文件转换为 HTML 或 XML 这类含结构化信息的文档格式，但它仍无法准确识别出文本中的段落和其它语义角色，并且在单词的识别上也没有考虑特殊字符的影响^[37]。Pdf-extract 同样是一款将 PDF 文件转换为 XML 的工具，它支持将文本分割成块、行和单词，但同样无法准确地识别出块及单词^[38]。文献 39 细地介绍了目前几种常用的解析 PDF 的商业软件，在文中详细地分析了每种软件的优缺点^[39]。每种工具都存在一些尚未解析的问题导致解析的效果欠佳，如 PDFtotext 它将 PDF 转换为纯文本，而不会保存其格式，很多时候它在提取文本时也是错误的，因此它在文本语义上的解析效果并不十分理想，同样在语义角色识别上存在问题的还有 PdfBox。PdfMiner 在针对被分隔符分割的段落时，无误识别正确的段落边界。Pdf-extract 存在正确提取下标和上标的问题，在很多情况下，该工具会把它们提取成单独的文本行，因为它们与所属文本行没有共享相同的基线，而其实这些上标与下标的文字应该属于它们所在的文本行，不应该被独立识别成行，类似的问题在数学公式，参考文献引用等角色上出现得较多。Pdftoxml, PdfMiner 无法将带连字符的字符串识别为单个字符，而把它识别为由连字符分割的多个字符。

1.3 论文主要工作

本文总结了此课题国内外研究现状，详细地对 PDF 的理论背景以及 PDF 解析相关技术做出了阐述，针对 PDF 文档中的两类主要元素：文本和矢量图形，详细地分析了其解析的关键因素以及影响解析效果的因素，并分别提出了具体的识别方法，针对块识别过程的优化设计了块的合并和块的排序算法。

我们的方法基于 PDFTron，它是一个跨平台处理 PDF 的库，在此框架的基础上，我们扩展了一些类集来实现我们的目的。对于解析出的文本块，我们设计了数据结构进行存储，使用嵌套的数据结构有利于快速访问到具体的块。采用自动化加人工调节的方式构造数据集 Json 文件，保存了其在 PDF 文件中的顺序及轮廓层次结果。

1.4 论文内容安排

本文主要研究对象是学术论文 PDF 结构解析技术，本文一共分为 6 章，论文安排如下：

第一章，对本文的研究背景及意义进行多角度地考虑与分析，分析了针对学术论文 PDF 解析的必要性。然后，简单介绍了当前国内外对本课题研究的现状，概述了两类主流的方法：基于学习以及基于规则，同时介绍了其它的一些研究方法以及现有的 PDF 解析工具。

第二章，对本研究相关技术进行介绍，概述了 PDF 文档对象、文档结构、图形元素、文本元素以及本文使用的框架 PDFTron。

第三章，讨论了文本元素解析关键因素以及影响文本元素解析效果的因素，提出了基于文本坐标规律的识别方法来对文本区域进行识别，对可能影响解析效果的问题设计了解决方案，识别过程中首先获取页面全局信息，然后依据文本行坐标的规律制定文本块合并的条件，对文本行进行合并以生成文本块。

第四章，讨论了矢量元素解析关键因素以及影响矢量元素解析效果的因素，提出了基于最大矩形区域扩充的识别方法，识别过程中先对 PDF 页面中的 PATH 进行还原，然后将 PATH 与周围的非正文文本进行合并以生成矢量图形块。

第五章，介绍了块的存储时需要用到的两个算法，合并算法主要针对于块间有重叠或交集，通过合并可以减少块间的冗余。排序算法主要是为解决页面元素的渲染顺序不一定严格按照从左至右、从上至下而带来的块的顺序不符合期望的问题。

第六章介绍了实验的设计与过程，包括自动化加人工调节的方式构造数据集、算法的处理过程，在实验中与现有开放的系统 PDFTron、PDFBox、现有算法进行了对比，并用效果图以及统计表格、对比图表从文本、矢量图像以及表格三方面描述了实验结果，并对实验结果进行了分析。

最后，总结了本课题涉及的相关工作，对本文所提出的基于文本坐标规律的识别方法与基于最大矩形区域扩充方法的创新点进行了总结，分析了实验中存在的不足之处，以及对后续工作的展望。

第 2 章 研究相关技术

本章主要是对本研究相关的技术进行介绍,从多角度介绍了 PDF 的文档对象、PDF 的文档结构、PDF 的图形元素、PDF 的文本元素,以及介绍了本研究实践中用以辅助的 PDFTron 框架。

2.1 PDF 文档对象

PDF 文档格式是 COS (Carousel Object System) 格式的子集, COS 树主要是由对象组成,对象被序列化为 PDF 文件中的内容,可以使用它们来构造 PDF。在本文在 2.5 节中介绍了 PDFTron 中对 PDF 文档的 COS 树的封装类 Obj。因此 PDF 文件实质上是一组链接对象,每个对象对应着具体的数字标号,对象间通过数字标号互相引用^[40]。对象可以是直接的,也可以是间接的,对象共有这几种类型: null, 布尔值, 整数, 实数, 名称, 字符串, 数组, 字典和流, 下面是对这几种对象的详细介绍。

(1) 空对象: 空对象代表缺失值, 如果将其实际写入文件, 则只是代表四个字符 null, 在 PDF 中不常见。

(2) 布尔对象: 它与面向对象编程中所代表的意义一样, 表示 true 和 false 的逻辑值, 并且是区分大小写, 在 PDF 中使用的是小写。

(3) 数值对象: 在 PDF 中支持两种不同的类型即整数和实数, 整数数字对象由十进制数字组成, 实数对象由带可选符号的十进制数字以及代表有符号实数值的前导组成。

(4) 名称对象: 它是一个字符序列, 以/字符后跟 UTF-8 字符串的形式写入 PDF 中, PDF 标准中对于字符串中的字符有规定, 如果要使用非常规字符, 都必须使用其特殊的编码形式。

(5) 字符串对象: 字符串序列化后表示为一系列 8 字节数据, 有两种形式, 一种为写在括号 () 内的文字字符, 另一种用尖括号 <> 括起来的十六进制数据, 通过使用反斜杠符 \ 进行转义。

(6) 数组对象: 它是包含在其他对象中的异构集合, 由方括号 [] 包含对象, 对象间用空格进行分隔, 数据中的元素对应 PDF 中的对象, 并且元素数量没有限制。

(7) 字典对象: 它是更高级别的对象, 也是 PDF 中最常见的对象, 它是键/值对的集合, 每个键是一个名称对象, 值可以是任何其他类型的对象。字典对象定义在双尖括号中, 在这些括号中, 键可以按任何顺序出现。并且可以在键/值对

之间放置换行符，提高可读性。如图 2.1 通过字典的形式，对类型、作者、资源中的字体都进行了描述。

```
<<
  /Type /Page
  /Author (Zhou Yongkai)
  /Resources << /Font [ F2 ] >>
>>
```

图 2.1 更具可读性的字典对象

（8）流对象：PDF 中的流是 8 位字节的任意序列，它的长度没有限制，并且可以进行压缩或编码。流对象的后面包含流属性的字典，使用单词 **stream**（在行尾标记之后）和 **endstream**（在行尾标记之前）来描述数据与其字典。流字典永远不会独立存在，它是流对象的一部分。在流字典中，至少包含 **Length** 键，它表示流数据中的字节数。流字典中的 **Filter** 键指定了在将原始数据包含在流中之前对原始数据进行了何种压缩或编码，其中很常见的有 **FlateDecode** 过滤器，**DCTDecode** 过滤器。如图 2.2 为流对象的示例，流对象的长度可以无限制，流对象将数据与字典分开，在字典中对类型、过滤器、数据长度等进行了描述，**stream** 与 **endstream** 之间包含了具体编码后的数据。

```
<<
  /Type /XObject
  /Subtype /Image
  /Filter /DCTDecode
  /Length 512
  /Height 64
  /Width 64
>>
stream
% DCTDecode编码后的512个字节的数据放在这儿.....
endstream
```

图 2.2 流对象的示例

2.2 PDF 文档结构

PDF 的文档结构可以从两方面来理解，物理结构描述的是文档的物理组织方式，它决定了对象会按照哪种方式存放在一个 PDF 文档中，PDF 页面中的对象是如何被访问以及被更新。逻辑结构指的是文档内容的逻辑方式。

2.2.1 PDF 物理结构

PDF 文档的物理结构描述了 PDF 将如何组织对象以实现对其的快速随机访问及更新，文档的物理结构由文件头（Header）、文件体（Body）、交叉引用表

（Cross-reference table）和文件尾（Trailer）组成^[41]。PDFTron 中文档 Trailer 的链接访问通过 SDFDoc 类实现,通过文档 Trailer 可以获取到 PDF 中的所有对象。如图 2.3 为 PDF 的物理结构图示:

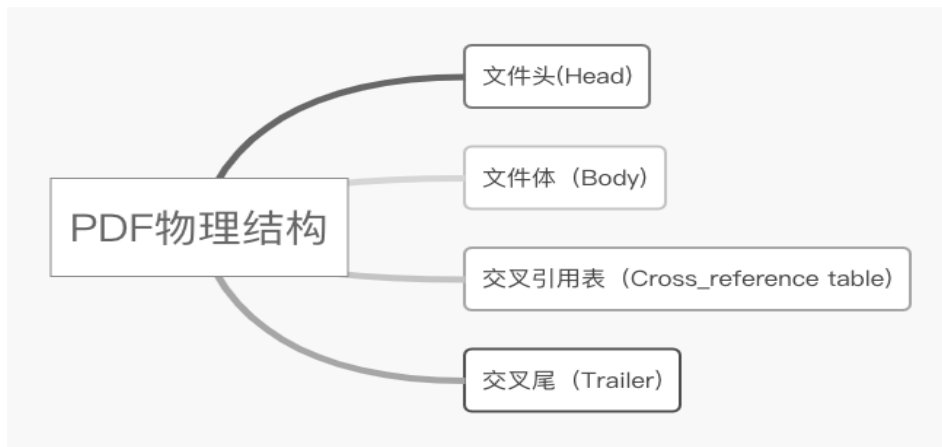


图 2.3 PDF 物理结构

文件头：它位于 PDF 文档的首行，从文件的字节 0 开始，定义了本文件遵守的 PDF 标准的版本号。PDF 的文件头由 8 个字节和一个行尾标记组成，文件所遵守的版本号被标识在这 8 个字节中。

文件体：由 PDF 中的对象按照组织方式组成的实际文档本身，是 PDF 文件的主要内容部分也是最重要的部分，文件中所有的对象都在文件体中被定义。对象的定义由 4 部分组成，对象序号用于唯一地标注一个对象，生成号代表 PDF 的修改次数，obj 与 endobj 可以看成是左括号与右括号，对象内容定义在这中间，如图 2.4 为对象的定义示例图。

```
3 0 obj
...这里可以是PDF规定的任意对象
end obj
```

图 2.4 对象的定义

交叉引用表:它为文件中的每个间接对象提供了从文件头开始的二进制偏移量，PDF 对象不必按照物理位置顺序排列，交叉引用表会用偏移+索引的形式保存对象的信息，PDF 处理器可以随时快速地查找并且读取对象，而不必处理整个 PDF 文档。但随着 PDF 文档的变大，在 PDF1.5 中，由于数据存储为标准流对象，引用了交叉引用流来提出一种新型的交叉引用存储系统^[42]。

文件尾：尾部是具有键和值的字典，该字典提供用于处理文档本身所必需的文档级信息。其中最重要的两个键是 size 和 root，size 键用于描述在尾部之前的

交叉引用表中有多少个条目，**root** 键的值是文档的目录字典，可以从这里开始查找 PDF 文件中的所有对象。

2.2.2 PDF 逻辑结构

PDF 文档由主体部分中所包含的对象组成，它具有 PDF 文件的文件结构，每个 PDF 文件都封装了固定的布局的平面文档的完整描述，包括文本、字体、字形、矢量图形，光栅图像以及显示它所需要的其他相关信息。PDF 将文档实现为对象的层次结构，并将它们组织为树和链接列表，可以对各种类型的内容进行封装，如果将 PDF 中的各个对象看成树，那么文档目录即为树的根，可以从而根开始用递归的方法遍历树^[43]。通过该对象，可以找到处理 PDF 页面以及其它内容所需要的所有其他的对象，大纲层次结构，文章线程，命名目的地和交互式表单的引用。PDF 文档中的大多数对象都是字典，文档的每个页面都由页面对象表示，页面对象是一个字典，其中包含对页面内容的引用^[44]。页面对象连接在一起并形成页面树，该页面树在文档目录中使用间接引用进行声明^[45]。两页 PDF 文档的典型文档结构如图 2.5 所示：

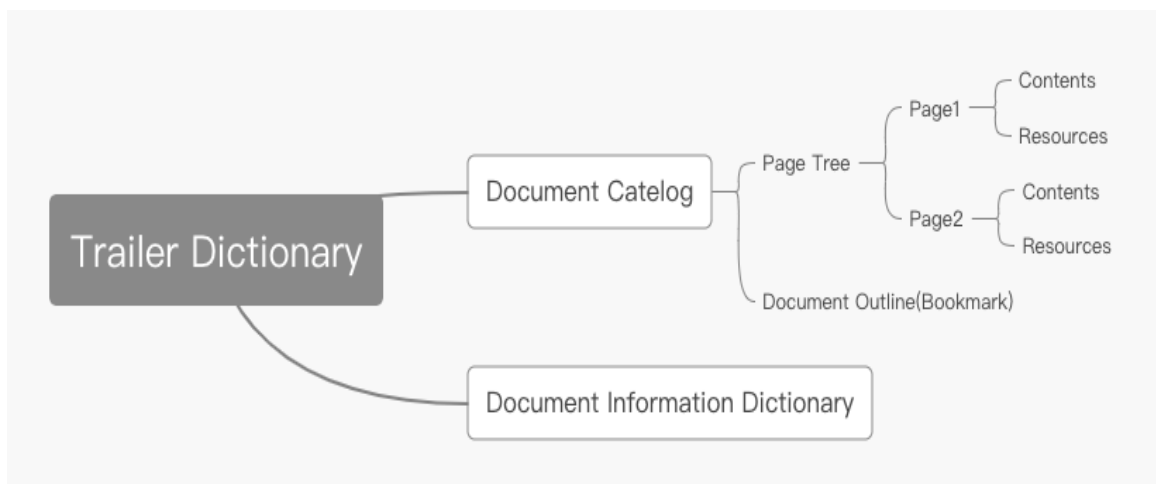


图 2.5 PDF 逻辑结构

Trailer dictionary: 它包含允许读取交叉引用表的条目，从而可以读取文件的对象，如前文所介绍的，该字典中所必须包含的条目为 **Size** 与 **Root**，同时也可以包含一些非必填项，如 **Info** 代表了文档信息字典，**ID** 条目代表了在工作流程中唯一地标识文件，如图 2.6 为 Trailer dictionary 举例。

```
<<
  /Size 323
  /Root 218 0 R
  /Info 216 0 R
  /ID [<64ff22189ceac627dfa2afec93deee02> <317928614d9711db612e000d937095a7>]
>>
```

图 2.6 Trailer dictionary 举例

文件信息字典 Document information dictionary: 包含文件的创建和修改日期, 以及一些简单的元数据。文件信息字典中的条目在表 2.1 中列出。

表 2.1 文件信息字典条目

键	值类型	值
/Title	字符串	文件标题
/Subject	字符串	文档的主题
/Keywords	字符串	关键字
/Author	字符串	作者名字
/CreationDate	日期	创建日期
/ModDate	日期	修改日期
/Creator	字符串	创建此文档的程序的名称
/Producer	字符串	其它启动与处理 PDF 的程序

文件目录 Document catalog: 是主对象图的根对象, 可以通过间接引用从中访问所有其他对象。目录是包含 PDF 文档根节点的字典以及有关首次打开时应如何显示文档的信息。目录中唯一需要的成员是对文档页面树的引用以及根的引用。

页面和页面树: 页面树定义了页面的顺序, 该节点可以有两种类型: 中间节点和叶节点。中间节点也称为页面树节点, 而叶节点称为页面对象。通过页面树访问文档的页面, 页面字典构建的页面树将用于绘制图形和文本内容的指令与这些指令使用的资源放在一起, 使用页面树而不是简单数组将页面链接在一起, 如图 2.7 为页面树图示。

2.3 PDF 图形元素

PDF 中的图形元素包含很多类型, 所有基本图形, 如直线、矩形、曲线, 或由基本图形与文字组合的矢量图片和表格等都属于 PDF 图形元素, 对于学术论文 PDF 的图形元素, 我们考虑版面中的位图、矢量图、表格、标线。PDF 图形使用与设备无关的笛卡尔坐标系来描述页面的表面, PDF 页面描述可以使用矩阵来缩放、旋转和倾斜图形元素。图形元素的构建过程与可缩放矢量图形(SVG)的构建类似, 首先定义或绘制形状 (或路径 PATH), 然后对形状描边与填充操作, 每个路径按照其在内容流中出现的顺序绘制, 新的路径都在已有路径后绘制。

2.3.1 PDF 图形内容流

1. 图形操作符

图形操作符是内容流中用来描述页面并生成输出到设备, 它使用一些一些路径构造、描边和线属性操作符以及操作数来构建, 每个运算符后面都有零个或多个操作数, 每个操作数都可以表示为标准 PDF 对象^[46]。路径 (PATH) 是 SVG (可

缩放矢量图形)中的一种元素,该元素有相应的指令来绘制直线、矩阵、贝塞尔曲线,可以结合使用直线,曲线等来制作各种复杂图形。PDF 的 PATH 可以被绘制、填充、剪切。描边和填充可以使用图形状态中设置的任何颜色,图形操作符包括 6 组,表 2.2 分为 6 组列出了 78 个图形运算符,这些出现在流对象中的操作符控制着打印页面上图形元素的外观,在 PDFTron 中操作符和操作数据都保存在 PathData 中。

表 2.2 图形运算符

操作符组	作用	操作符
图形状态操作符 Graphics state operator	更改当前图形状态 (当前变换矩阵,当前颜色,笔触宽度等)	w J j M d ri i gs q Q cm CS cs SC SCN sc scn G g RG rg K k
路径构造运算符	构建线条,曲线和矩形。	m l c v y h re
路径绘画操作符	描边和填充路径,或使用它们来定义剪切区域。	S s f F f* B B* b b* n W W*
其他绘画操作符	底纹和内嵌图像	sh BI ID EI Do
文字操作符	选择并显示各种字体和方式的文本	Tc Tw Tz TL Tf Tr Ts Td TD Tm T* Tj TJ ' " d0 d1
标记内容操作符	用于划分流的各个部分	MP DP BMC BDC EMC BX EX

路径可以由多个子路径构成,它由 3 个运算符来移动点:m 代表 moveto, l 代表 lineto, c 代表 curveto,每一个新路径以 m 运算符开头,使用 m 运算符移动到新路径的开始点,使用 l 运算符按直线移动至结束点,使用 s 操作符(stroke)在页面上将线条进行描绘,除了直线,PDF 中还可以绘制曲线,曲线由四个点(起点和终点)和两个控制点定义,两个控制点定义曲线在起点和终点之间的形状。路径的填充和描边也是支持的,图 2.7 为 PATH 绘制命令示例:

```
2.0 W 设定线宽度
0.75 g 设置颜色
250 250 m 移至开始点
350 350 450 450 550 250 c 画曲线
450 250 350 200 y 第二个控制点
h B 闭合并描边路径
```

图 2.7 PATH 绘制命令

第一行 **w** 运算符用于设定图形状态的线宽度,第二行 **g** 运算符用于设置颜色,第三行 **m** 运算符用于移动至开始点,第四行 **c** 运算符用于画曲线,第五行使用了类似于 **c** 的运算符,只是第二个控制点和端点是一个相同的端点,因此只需要 4 个操作数,末行 **h** 运算符用于手动闭合路径, **B** 运算符则设定填充并描边路径。

2.3.2 PDF 光栅图像

常见的光栅图像为 JPEG,PNG,GIF 或 TIFF 等,PDF 文件通常将光栅图像存储为单独的对象 (XObject), 其中包含该图像的原始二进制数据。PDF 中若需要创建及显示光栅图像, 首先必须创建图像字典并添加图像数据, 然后在资源字典中添加对图像字典的引用。光栅图像并不是仅由简单色彩空间中的路径组成的内容, 它属于外部引用资源。光栅图像与其它常见的 PDF 数据结构一样, 它也是由带有相关流的字典所表示, 图像字典描述了图像的相关属性, 而在资源字典中添加了图像字典的引用, 并且在流中包含了图像数据, 通常, 光栅图像是可以直接以页面描述来作为嵌入式的内嵌图像, 为了达到压缩的目的, PDF 支持多种图像过滤器, PDF 文档常用编码方法所对应的解码器包括 FlateDecode、CCITTFaxDecode、JBIG2Decode、DCTDecode 等^[44]。被嵌入的 PDF 图像通常不是这些格式, 而是与该数据所使用的色彩空间及有关图像的信息一起存储为像素的二进制数据^[47]。有 2 个用于绘制光栅图像的图像命令 (ID 和 DO)。现有的解析工具一般都提供了光栅图像提取的接口, 因为其图像数据是做为一个整体进行存储, 在生成绘制光栅图像时, 使用 ID 命令允许将二进制图像数据嵌入命令流中, DO 命令将图像存储在 XObject 或 XForm 类型的单独的 PDF 对象中。JPEG 文件是可以直接放入 PDF 的唯一标准图像格式, 只需从文件中读取数据流, 然后将其写入 PDF 中的流对象的值即可。图 2.8 为光栅图像存储在/Xobject 页面资源字典中的示例。

```
<<
  /Type /XObject 代表这是一个XObject
  /Subtype /Image 代表这是一个图片
  /ColorSpace /DeviceGray 图像的颜色空间
  /Length 8 以字节为单位的流长度
  /Width 8 以像素为单位的图像宽度
  /Height 8 以像素为单位的图像高度
  /BitsPerComponent 1 每个组件使用的位数
>>
stream
@`pxxp`@ 图像数据
endstream
```

图 2.8 光栅图像在资源字典中的定义

2.3.3 PDF 矢量图像

PDF 实际上并没有矢量图像的概念,与栅格图像一样,这是另一种 XObject,称为表单 XObject^[46]。表单 XObject 是一个内容流,PDF 中有相应的表单字典,所有放入页面内容流中的任何内容,都可以放入表格 XObject 中,因此可以在其他位置重用。与图像 XObjects 一样,在 PDF 中添加表单 XObject,首先需要创建一个表单字典并添加数据,然后在资源字典中添加对表单字典的引用。表单字典中有如下四个特殊键:

(1) Matrix: 标准转换矩阵,所有 Xobject 在绘制与生成时,在调用内容流中都会使用到该转换。

(2) BBox: 表示该 XObject 的边界框的阵列,一般它的大小为图形实际内容的大小,能完整地包含 Xobject 的内容。

(3) Resources: 引用内容流中需要用到的额外资源的地方。

(4) FormType: 可选,没有实际用途。

2.4 PDF 文本元素

PDF 文本由一系列文本运算符包含起来的内容和相关参数组成。PDF 文档的字体文件嵌入在文档中,因此字符形状始终可用,因此,这也保证了 PDF 的设备独立性,在生成文本也可以使用复杂的图形效果。通常,在 PDF 文档中会编码足够的信息以允许文本提取。在 PDFTron 中,文本为 e_text 的 element,它对文本的相关属性都进行了封装,文本元素中几个需要理解的概念为字体对象、文本矩阵和文本显示。

2.4.1 PDF 字体对象

PDF 中的字体对象是对字体的描述,如果描述对象为文本中的字符,则为非嵌入字体,另一种则为嵌入字体。字体对象描述了文本的相关信息,其中包含字体、位置等格式信息,字体实际上就是一堆字形的集合。一般的,根据字体格式类型可以分为: Type 1, TrueType, OpenType, 而在 PDF 中,字体类型为: Type 0, Type 1, Type3, TrueType, CIDFont^[48]。文本字符串中,字符用以编码映射字形于当前字体的字符代码(整数)显示^[49]。文本运算符指定要绘制的字形,这些字形由字符代码序列表示,在字体词典和相关的数据结构中保存了字体相关信息,尤其是字形。PDF 对于没有嵌入的字体,会根据字体名称查找系统下是否存在该字体,不存在的话会选择类似字体,因此,PDF 中的文本在不同设备上的显示由系统下字体文件决定。与图形状态类似,表 2.3 为字体对象的文本状态参数以及运算符。

表 2.3 文本状态参数及运算符

状态参数	描述	运算符	作用	初始值
Tc	字符间距	charSpace	设置字符间距	0
Tw	字间距	wordSpace	设置单词间距	0
Th	水平间距	scale	设备水平缩放比例	100（正常间距）
TL	行距相对值	leading	设置行距。	0
Tf, Tfs	字体大小	font, size	设置字体大小	必须指定
Tmode	渲染模式	render	设置文本渲染模式	0

2.4.2 PDF 文本位置

文字空间定义了文本的坐标系，从文本空间到用户空间的转换确定了文本在页面上旋转的位置。通常文本字符串中第一个字形的原点位于文本空间的原点，文本矩阵描述了字体和位置信息。在操作符中，Tm 操作符可以设置文本矩阵，它共有 6 个参数，即[Sx 0 0 Sy Tx Ty]，其中参数 Sx 和 Sy 代表字体的宽度和高度，而参数 Tx 和 Ty 则代表了当前的水平和垂直位置。具体涉及到两个矩阵：

(1) 文本矩阵，用于定义下一个字形的转换。它通过文本位置和文本显示运算符来更改。

(2) 文本行矩阵，它是当前行开头的文本矩阵的状态。因此，文本行可以通过使用操作符来来在文本行间移动，而无需手动去定位行首的位置。

这两个矩阵定义了从文本空间到用户空间的转换。用于修改文本位置的文字定位运算符在表 2.4 中列出。

表 2.4 文字定位运算符

操作数	运算符	功能
x,y	Td	将文本位置移动到下一行，偏移(x,y),参数以未缩放的文本空间单位表示
x,y	TD	将文本位置移动到下一行，偏移(x,y),同时设置 Leading 值为-y
--	T*	将文本位置移到下一行的开始位置
a,b,c,d,e,f	Tm	将文本矩阵和文本行矩阵设置为[a b 0 c d 0 e f 1]

2.4.3 PDF 文本显示

PDF 中的文本是由页面中内容流的文本元素来表示，PDF 内容流为 8 位字节的序列，包含在关键字 stream 与 endstream 之间，所有由原始二进制数据组成的任何类型的内容都由流表示。由于流也是间接对象，因此流将由 obj 和 endobj 关

键字括起来。文本部分被定义在运算符 BT 与 ET 之间，BT 代表起始文本块，ET 代表结束文本块，文本元素将指定页面中应在哪些位置绘制字符，以及使用选定的字体资源的编码来指定字符。PDF 中的字体对象可以描述字体的特征，PDF 支持多种字体。文本有许多属性，最重要的三个为字体、文本大小和放置文本的位置。在文本操作符中，TF 操作符用来指定字体资源的名称，即当前资源字典的字体子字典中的条目。Tj 操作符用于在当前位置显示字符串，并带有针对单个字形位置的调整。Tf 运算符用来选择字体名称，图 2.9 为文本对象在流中的定义。

图 2.9 中第一行给出对象编号 2 以及版本编号 0，属性字典仅包含一个长度键，该键显示从 stream 之后的行的开始到包含 endstream 的行的开始的字节数。由于流由可显示的文本组成，因此在页面标记运算符 BT 和 ET 中将其括在括号内，以表示“开始文本”和“结束文本”。以“/F1”开头的行表示要查找并加载 12 点大小的 1 号字体。下一行以“72 712 Td”开头，这意味着将文本放置在用户空间中的 $(x, y) = (72, 712)$ 处，后跟显示文本运算符 Tj。

```
2 0 obj
<<
/Length 39
>>
Stream
BT
/F1 12 Tf
72 712 Td (A short text stream.) Tj
ET
Endstream
endobj
```

图 2.9 文本对象在流中的定义

2.5 PDFTron 框架

PDFTron 是一个 PDF 编辑开发类库，它能够满足多样化的需求并且提供了跨平台的高质量工具包，开发借助它所提供的 API 可构建可靠的、跨平台的应用程序。PDFTron 所提供的 SDK 支持所有主要平台（JAVA，.NET / WPF，C / C ++，Python，PHP）且适用于主流操作系统（Linux，Windows，Mac，Android，iOS，WinRT，WP8），支持创建、查看、编辑、优化、查看和注释 PDF、MS Office 等 30 多种文件格式。PDFTron 对 PDF 的一些常用功能需求上提供了很好的支持，为 Web、移动应用、桌面应用和服务器应用嵌入优秀的文档处理和编辑功能。它主要提供了以下几方面的功能。

（1）页面操作：支持对页面的更改，提供对 PDF 文档追加、拆分、添加添加，删除，裁剪，重新排序，旋转和导出 PDF 页面的功能。

(2) 编辑页面内容: 支持对内容的编辑功能, 以便可以改善其数据或重新使用其数据, 提供在现有页面删除, 插入或重新放置特定内容 (例如文本, 图像, 路径, 书签、和基于某些属性或页面上的位置信息等) 的功能, 支持修改现有图形元素的属性如字体、字符间距、填充颜色等。

(3) 附加多个 PDF: 从动态和静态文档的组合中组装新文档, 可将多个 PDF 文档插入/复制/合并到一个文档中。

(4) 完全文件控制: 通过其低级 PDF 对象模型 (COS / SDF API) 访问所有文档信息。修改图像, 图形属性, 字体, 编辑文本, 元数据, 注释, 表单字段数据等等。

(5) 程序化和互动式: 大规模自动化文档处理, 编辑和组装, 或将 UI 小部件嵌入应用程序中以根据需求定制。

PDFTron SDK API 分为多个名称空间, 其中最重要的是 PDF、SDF 和 Filter, 对于 PDF 内的数据提供了抽象的接口。PDF API 是一个简单的高级 API, 用于处理 PDF 构造, 例如页面, 交互式表单, 书签, 页面上的图形元素等。SDF API 是一个强大的一个强大的底层 API, 用于处理 PDF 文档的各个方面。Filter API 是一个强大的实用程序 API, 可处理 PDF 文档中使用的各种压缩和加密方案。本论文中需要用到的几个主要的类有:

(1) `com.pdftron.sdf.Obj`: 是所有 SDF/Cos 对象的具体类, 使用复合设计模式实现了对象层次结构。可以使用 Obj 接口调用任何“派生”对象的成员函数, 2.1 节对 PDF 的几种文档对象展开了介绍。

(2) `com.pdftron.sdf.SDFDoc`: 是表示 SDF.Obj 节点图的低级类, 可用于构建高级文档模型, 如 PDF 或 FDF (表单文档格式), 可以使用它提供的默认构造函数从头开始创建 SDF, 可以访问文档 Trailer 获取每个文档中的起始 SDF 对象 (根节点), 通过文档 Trailer 的链接, 可以访问文档中的所有低级对象, 2.2 中对文档的物理结构展开了详细的介绍。

(3) `com.pdftron.common.Matrix2D`: 是 PDFTron 用来表示仿射变换的矩阵, 它指定两个坐标空间的关系, 通过它可以获得仿射变换中矩阵的 6 个参数, 以实现坐标转换。

(4) `com.pdftron.pdf.annots`: 支持添加和编辑 PDF 批注、使用户在不更改原始作者内容的情况下对 PDF 进行标记和评论, 注释的形式可以是超链接, 文档内链接, 图章, 文件附件, 文本, 自由文本, 线, 圆, 正方形, 多边形, 高亮, 曲线等, 用户可以记笔记、进行注释, 突出显示, 添加形状。

但 PDFTron 所提供的接口中, 对于文本的提取, 它无法识别到精确的块和段落, 在更细粒度的 API 中, 对于文本行的提取, 也会因为字符的边界框的交叉而导致识别不准确, 并且对于文本行的解析顺序, 会因为与图片的干扰而导致与人

为期望的不一致。对于图像的提取，PDFTron 对于光栅图有不错的表现，但是对于不是作为整体存储的矢量图而言，它无法对其进行识别，并且没有提供对表格提取的有效接口。本文基于 PDFTron 辅助实现，在它的基础上扩展了许多类集，并对页面、文本、图像的相关信息进行了封装和过滤，利用这些信息制定规则，对文本区域以及非文本区域进行识别。

2.6 本章小结

本章节从多方面介绍了 PDF 文档，首先介绍的是 PDF 中的文档对象，然后概述了两种 PDF 文档结构，物理结构以及逻辑结构，物理结构介绍了对象如何按照组织方式组成实际文档本身，逻辑结构将 PDF 对象看成树。然后介绍了 PDF 中的图形元素，阐述了图形流的构建以及光栅图像与矢量图像的区别。接下来介绍了文本元素，对其字体对象、文本位置、文本内容进行了描述。最后介绍了本课题借助的 PDF 解析框架 PDFTron。

第 3 章 文本区域识别方法

本章节对 PDF 文本解析的关键因素以及影响 PDF 文本区域解析效果的几方面问题进行了分析，提出了基于文本坐标规律的识别方法，首先获取文档的全局信息作为后续判断的条件，由于文本块是由文本行组成，因此首先识别文本行并提取其特征，在本文中使用文本行的坐标规律作为文本块扩充的条件，并按设计的数据结构存储提取的文本块。

3.1 关键因素及影响分析

3.1.1 文本解析关键因素

PDF 中的文本对象包含了文本内容，同时描述了文本的字符间距、字体大小、字形缩放、文本呈现模式等，由 BT 操作符开始，以 ET 操作符结束，由文本运算符以及相关参数组成。文本的位置信息由文本矩阵描述，由 Tm 操作符完成设置，Tm 操作符有 6 个参数，Sx,Sy 描述字体的宽和高，Tx,Ty 定义当前所在行的水平位置和垂直位置，文本矩阵表示为[Sx 0 0 Sy Tx Ty]。文本的字体类型和编码信息存储在字体字典中，尤其是字形会通过保存在字体字典中而嵌入到 PDF 文件中，这种被称为嵌入字体，当字体程序（实际的字体文件）中未嵌入 PDF 文件时，字体字典中也会提供用于替代的信息。从字体文件中提取出用于构建字体字典、字体度量和字体编码字典的详细信息，具体过程取决于字体格式。通常，字体字典中包含大量的信息，使得在任何，文字的显示效果一致，由其是打印的场景，因此会让 PDF 文件变得更庞大。PDF 对于未嵌入字体，会先检查系统下是否有该字体的相关信息，如果不存在，则按照替换原则去选择替换信息中的类似字体，因此，带来的一个难题是，相同的 PDF，在不同设备下，如果系统中的字体文件不一致，则非嵌入字体的文字其显示效果也不一致。提取文本的另一个难题是需要内容流中重建文本运算符，这是由于运算符可能会拆分文本以进行字距调整或对齐，并且文本行尾的连字符“-”会打断字符流。

对于 PDF 文本的解析，需要考虑以下几个关键步骤：

- (1) 单词解析
- (2) 文本行解析
- (3) 渲染顺序
- (4) 矩形框边界校正
- (5) 块的构建

3.1.2 文本解析影响分析

PDF 文档是由一系列指令流来描述其格式,所有对象在 PDF 页面上的绘制都由指令流来定义。PDF 文件包含用于绘制形状,图像和文本的全部命令,它实际上是矢量图形的一种形式。PDF 是一种基于布局的格式,它指定了所有字符的字符和位置等特征,理想情况下 PDF 可以带标签,但绝大多数下它并不包含文本的语义结构单元例如单词、文本行、段落等结构,对于文档中如标题、题注这些帮助文档理解的结构并不会被标记,因此,文本数据的存储只是表示在页面具体位置上绘制的字符,而不是具有语义的段落、文本行甚至是单词。所以,当在生成 PDF 格式的文档时,大部分语义内容便已经丢失,所有的文本结构都转变为页面中指定区域的字符。PDF 的设计初衷并不是数据输入格式,而是输出格式,也正是这样的设计令对显示的结果文档做精细地控制十分方便。文本在 PDF 文档中是作为一组字符串对象来存储,文档中的所有对象都会包含特征信息。字符的视觉呈现即字形,它的构建由坐标位置、字号、字体以及其他的特征信息决定。值得注意的是,PDF 文档中同样的字符串对象不一定在不同文件中保持一致,其中的主要原因是 PDF 在创建之初所使用的工具的不同,或者是 PDF 内容使用的是非嵌入字符,而不同文件字体字典中使用的备用字体信息不一致。

影响 PDF 解析准确度的因素,主要有以下几方面:

(1) 单词解析:单词解析是一项具有挑战性的任务,有多种原因使得无法找到正确识别的单词。PDF 文件格式非常灵活,使用回车符、空格符、制表符或者换行符作为 PDF 数据间的分隔符,通常情况下,单词间使用空格作为分隔符,文本行是由多个空格分隔的单词所构成,而单词与单词间的间距在一行中和行与行间都会发生变化,很多时候,为了实现在格式上的对齐,需要调整字距,因此 PDF 文本行的单词间会包含多个空格,没有用来确定单词间距的固定规则,甚至单词中的字母间也可能包含有多余空格。通常情况下,单词中的每个字符的框会相邻,但这也并不固定,如前文所描述的,PDF 中的字符在实际转换时也会因为特殊符号被打断而识别成多个单词。在提取和解析 PDF 文档中的数据与相关信息时,需要非常细致以避免单词提取可能带来的误差。

(2) 文本行解析:正常情况下,一个文本行代表同一行内的文本具有相同的字体特征和属性,但并非是一种标准,从另一角度看,同一行内的文本具有相交的矩形框,但对于表格而言,位于表格内且 y 坐标相同的文本不能被识别为同一行。因此,许多现有的解析工具将文本行看成是一系列具有相同字体特征的文本作为提取文本行的依据会带来误差,因为文本行中的每个单词、甚至是单词中的每个字母都可能使用不同的字体,对于文本行中的特殊字符、公式等也使用与普通文本不同的字体特征,因此这些不同字体的单词、字母、特殊符号会被当成单独的文本行,从而导致文本行识别的不准确。

(3) 指定位置提取：PDF 文档中的文本可以被存储为单独的字符、行的一部分或者整行，可以按照不同的方式排序。由于文本可以是单独存储，因此我们在需要对页面进行解码并从页面区域中提取文本时，因为 PDF 的工作方式，除非解析整个页面，否则无法确定页面的任何位置。这也给提取文本带来了一定的困难。

(4) 矩形框边界校正：由于书写规定，大小写字母具有不同的对齐线，特殊字符也占据与普通字母不同高度的外框，因此这是一项更有难度的任务，因此我们需要根据对字符串中的每个字符所占高度进行投票，将最高点与最低点连接形成矩形框的水平方向的边线。

(5) 渲染顺序：正确识别单词的阅读顺序是至关重要的，当需要提取单词周围邻近信息时，单词顺序是一个很好的依据，通过位置可以很容易地得出行间单词顺序，但行间顺序并不一定正确，尤其是在版面为两列的 PDF、以及有图片占据整个页面宽度的情况，行的顺序在两列之间会交错，因此提取的相邻两行在页面上的布局不一定相邻，对于文本块的合并会带来影响，如图 3.1，页面从右列的顶部开始，在 1 到 7 行之后，它将继续呈现左列的文本，在 8 到 51 行之后，它将继续在右列中呈现其余的文本，此时，渲染编号为 52。但是，预期的渲染顺序应为从左到右，从上到下。

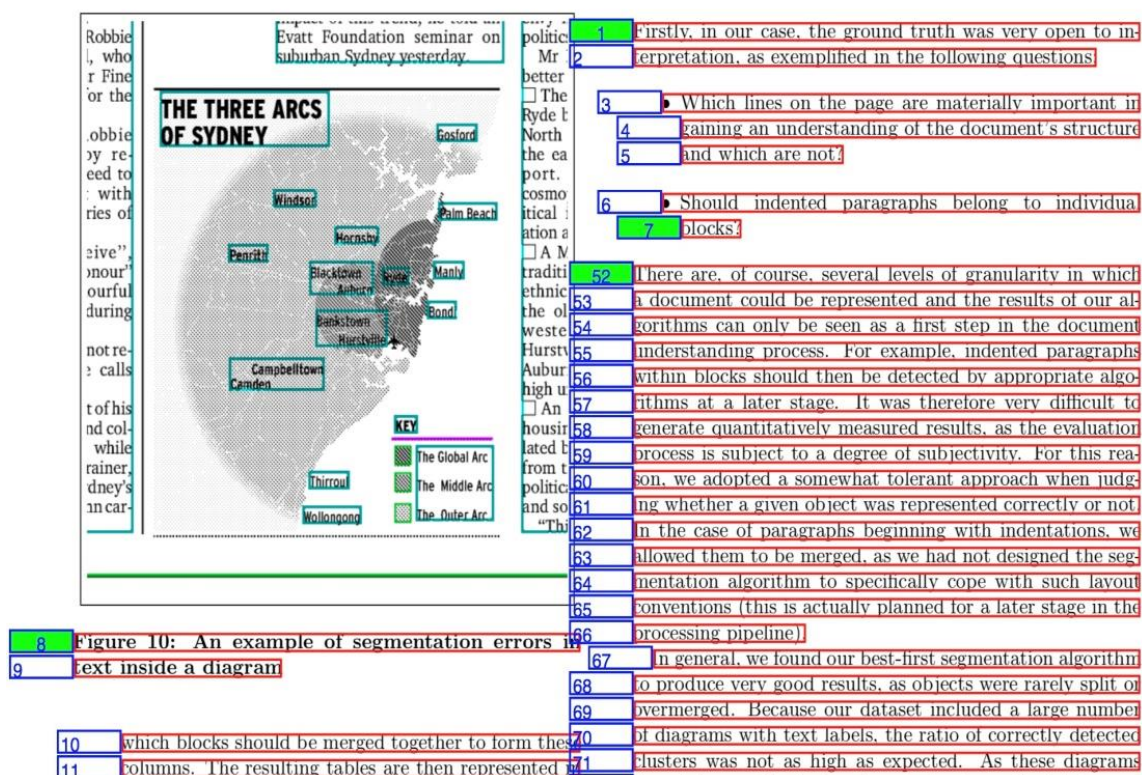


图 3.1 文本行的渲染顺序有误

(6) 块的构建：如单词可能会被特殊字符打断一般，同一文本块中的文本也可以由公式打断，公式可以是行间公式或者行内公式，在很多情况下也会由于版

面的原因被打乱，如文本流已按正常顺序到达页面或一栏的末尾，当在下一页或下一栏继续渲染文本块时，会构建成不同的块。当文本行渲染的顺序不符合期望时，文本行的顺序在两列之间会交错因此导致块的构建被终止而产生多个块。

3.2 基于文本坐标规律的识别方法

我们的目的是对文本区域进行识别并提取出文本块，由于文本块由文本行构成，因此首要任务是解析 PDF 文件中的文本行，我们的算法先初始化一个文本行并将它初始化为文本块，根据文本块中文本行间的规律，制定文本行合并的条件，利用迭代器逐行迭代文本，如果下一行文本满足文本块合并条件，则扩充文本块矩形使其包含下一文本行，如果下一文本行不满足文本块合并条件，则保存当前文本块，并将下一文本行坐标定义为新的文本块。图 3.2 为具体的流程图。

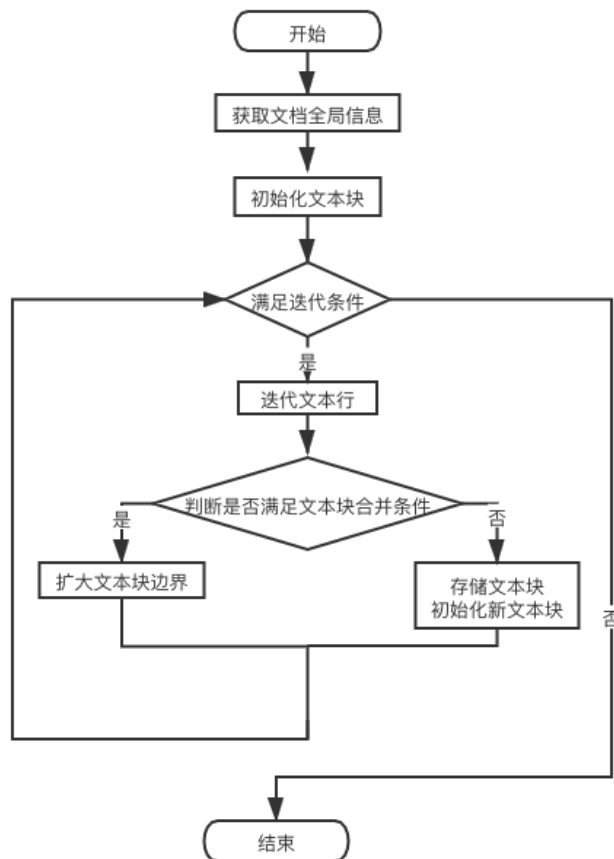


图 3.2 文本区域识别流程图

对于解析出的文本块，我们设计了数据结构进行存储，要保存的文本块包含字体、字号、坐标、是否斜体、是否加粗、所处栏等信息，我们使用 map 来进行保存，PDF 中每页包含许多文本块，因此使用 list 来保存这些 map，而一个 PDF 文档包含很多页，因此外层再嵌套 list 来保存这些页。这样嵌套的数据结构有利于快速访问到具体的块。图 3.3 为详细的数据结构。

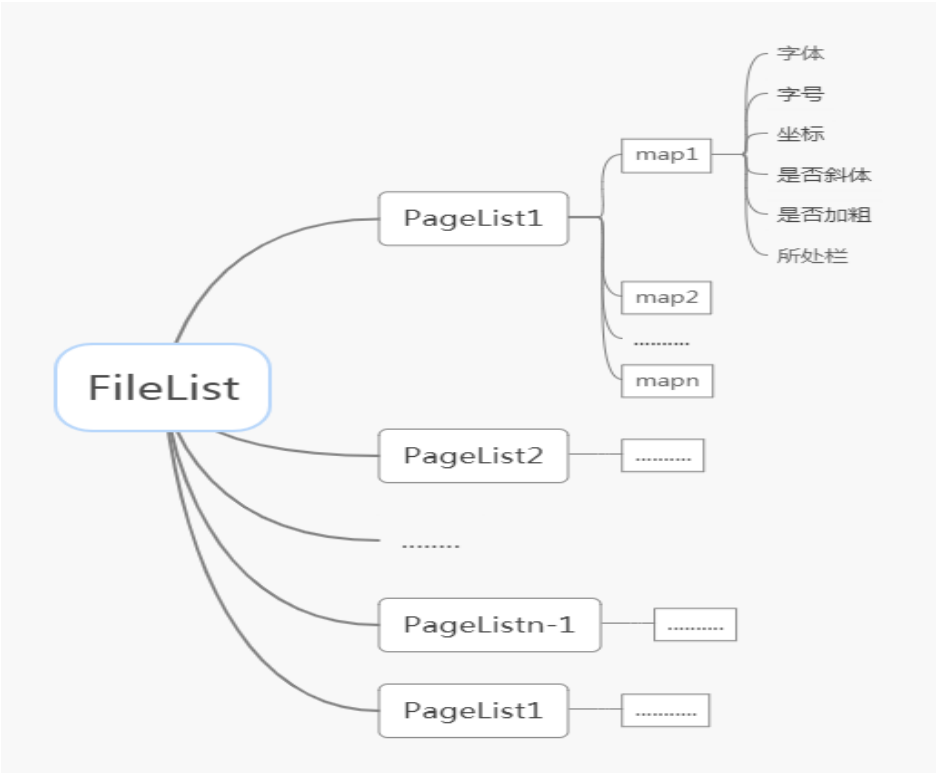


图 3.3 数据结构图

3.2.1 全局信息获取

PDF 学术论文中包含文档的全局信息，通过 PDFTron 的接口，我们可以获得文档的页数、页面的宽与高、页面的 cropbox 值（定义了页面内容的裁剪区域，Adobe Acrobat 软件使用这个来显示和打印页面），而一些具有语义结构的信息 PDFTron 无法提供，这些信息包括：页面内容起始位置坐标、正文行距、是否含有页眉、页面的栏数等。这些信息在后续的识别算法中需要用到，因此，我们针对需求对文档的全局信息进行了过滤和再封装，只保留了对区域识别有效的属性。图 3.4 为本系统中全局信息的表示情况。



图 3.4 全局信息类

com.pdftron.pdf.PDFDoc 是描述单个 PDF（便携式文档格式）文档的高级类，使用它提供的 getPage()方法访问文档的第一页，遍历其中的每一文本行，计算其

与上一文本行的间距，取所有间距的众数即为正文的行距。获取文本行的宽度，取出现的所有宽度的众数即为正文的栏宽。由于本实验主要针对学术论文 PDF，通常情况下文本版面为一栏或两栏，因此栏数的判断结果可以简单分为两种，通过比较栏宽与页面宽度的大小，若小于页面宽度的一半，则判断页面为两栏，否则为一栏。

PDFDoc 提供的 `getPageIterator()` 可以迭代访问 PDF 中的所有页，通过它遍历 PDF 文档每一页并提取第一个文本行，比较每页第一个文本行去空格后文本字符串内容是否相等，并判断其坐标是否相等，若两个条件均满足，则判定页面含页眉。判断页面的起始位置，需要考虑文档是否有页眉，判断的方法为遍历 PDF 文档每一页，若文档有页眉，则第二个文本行坐标为页面起始位置坐标，若文档无页眉，则第一个文本坐标为页面起始位置。

3.2.2 文本行坐标获取

英文学术论文版面中的文本行由单词构成，其中的字符包括英文字母、数字以及希腊字母，在这中间，英文字母所占据的版面篇幅最多，所以可以将它的尺寸作为版面字符的尺寸。英文字母具有大小写形式，排版与书写一样对齐三条标准线：顶线、基线和底线，因此根据字母中心所处的位置分为三种类型的字母，并且其中基线型的最多^[29]。三种类型的文本如图 3.5 所示：

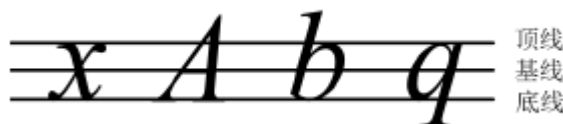


图 3.5 三种类型的文本

一般来说，一个 PDF 文档中普通正文由于使用同一种字体及字号，因此具有相同大小的外接矩形框，而特殊字符、上标、下标则不一致，它们的外接矩形框会明显大于实际占用的面积，实际中字符并不相交且这些矩形框之间却经常发生相交的情况。投影法在文本行提取中经常被用到^[50]。行提取时，经常会将一行分割成多行，这是因为行内字符可能存在着上下的位置关系，由于排版的原因，导致行与行之间在水平方向上存在重叠的部分，或者版面存在文本围绕着图像的现象，往往特殊字符所占据的空间也和普通文本不一样，导致不能准确地提取到行，如图 3.6，当单词间存在连字符“-“或者由于排版的因素存在空格“ ””时，会将英文单词一分为二，这对单词的提取造成了干扰。在字符相关信息中，字体属于其中一部分，字符显示内容的表现形式是依赖于该字符所使用的字体，因此，对于文本提取而言，首先应该提取字体信息。页面中的资源都会存储在页面对象中的资源字典里，字体便属于页面的资源。字体的基本信息都保存在字体字典对象及字体描述符对象中，这些基本信息包括字体名称、字体类型、字形宽度和字体

外接矩形框。然后第二步是对页面内容进行提取，它们一般由在页面的内容流来存储。

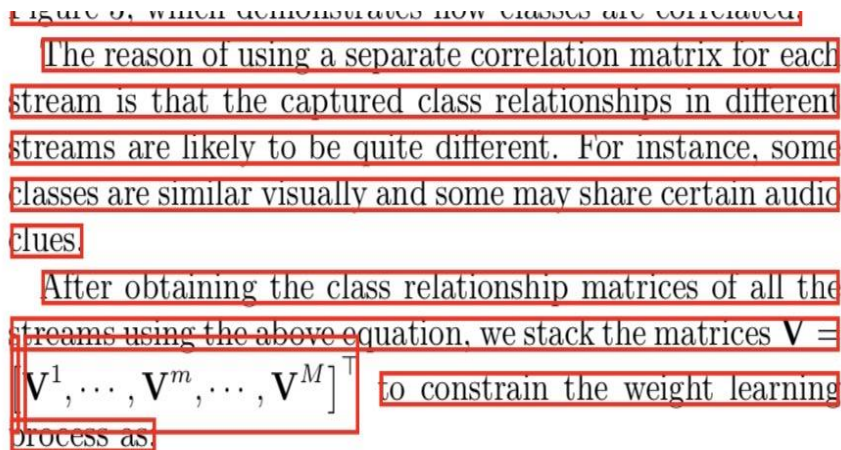


图 3.6 PDFTron 提取行效果

PDFTron 提供 `ElementReader` 类来读取元素，类型为 `e_text` 的元素即对应 `Tj` 元素，每一个 `e_text` 元素代表着一个文本行，一般而言，PDF 中的文本行都会使用相同的字体，如果行中出现不同字体的单词，则不同字体的单词将会被提取成一个单独的文本行，同样地，如果有一个单词出现不同的字体，则不同的字母将被提取成一个单独的文本行，在 PDF 中，还会遇到文本行中包含多个用空格分隔的单词。因此不能保证尝试使用 `ElementReader` 从 PDF 文档提取文本数据不能按预期的顺序（读取顺序）返回数据。`TextExtractor` 类是 PDFTron 用于分析 PDF 页面，并提取在给定区域内可见的单词和逻辑结构，它提供了相应的 API 以逐字、逐行地提取推断的逻辑结构，针对每次行、每个单词或者一个字形提取样式及定位信息，它将组合单词、行和段落、删除重复字符串、重建文本阅读顺序等。提取单词、行的表现不错，但是对于一些复杂排版的 PDF，由于同一文本块中上下行因为字体样式不同，导致提前进行了分块，因此文本块的提取需要重新设计算法。它提供了嵌套类 `TextExtractor.Line`，利用它可以读取到文本中的每一行，通过 `getStyle()` 方法可以得到文本行的样式，如文本行的 `BBox`、第一个单词、单词数等。在解析页面内容流后得到的 `BBox` 是由对应字体的 `BBox` 而来。`getStyle()` 中文本行的边界框取法为第一个字符及最后一个字符的坐标所连成的矩形框，也即文本行的边界框由第一个字符和最后一个字符决定，但这带来的问题也显而易见，若这两个字符为顶线型字母、底线型字母或特殊字符，则此行的边界框将不同于大多数文本行，大概率为大于普通文本行的高度，如图 3.7。

图 3.7 中用每一行文本左边绿色的注释框中打印出了本行的行高，右边标注序号为 1 的文本行为章节名，其行高为 13.98，其余普通正文行高为 8.97，而序号为 2 和 3 的文本行由于特殊字符的影响，行高分别为 12.01 与 10.12，与普通正

文行高不同。因此我们需要使用更细的方法，通过读取到每一个字符的高度，将高度的众数定义为文本行的高度，以提高准确率。

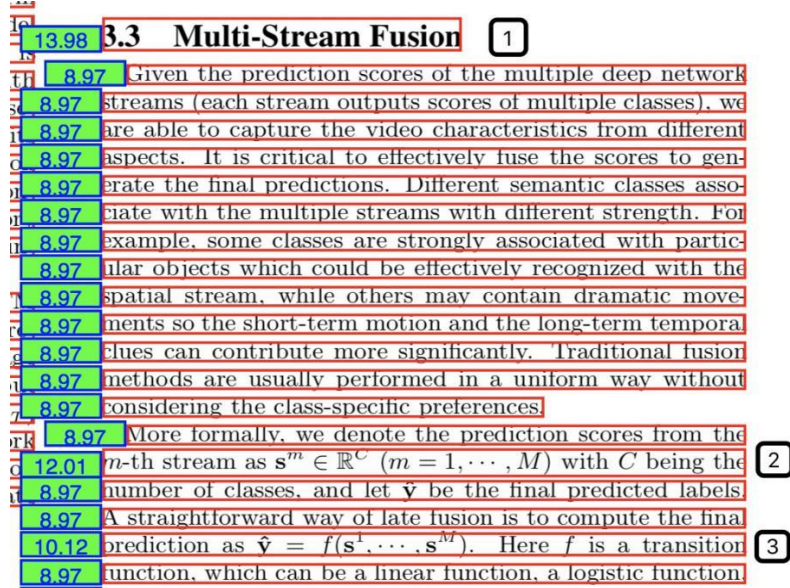


图 3.7 PDFTron 提取行高

3.2.3 文本块生成

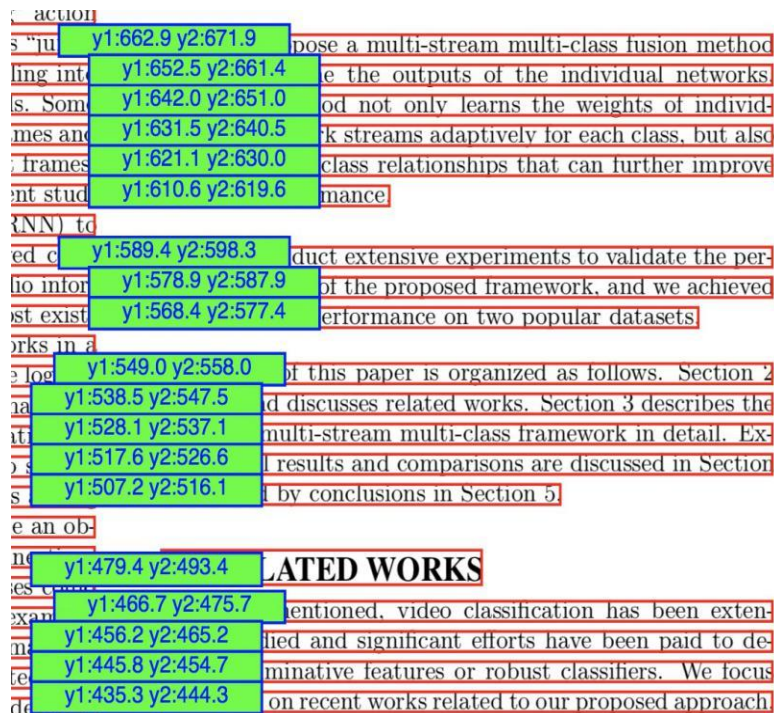


图 3.8 文本行 y1 与 y2 坐标值

为了便于直观理解，我们将文本行的坐标值 $y1$ 与 $y2$ 打印，如图 3.8 中所打印的注释，我们发现坐标值间存在规律，处于同一文本块的相邻两行，当前行的 $Y2$ 值小于等于上一行的 $Y1$ 值。

因此，我们可以根据这一特征来制定文本块合并的规则。遍历第一个文本行，并将其坐标初始化为第一个文本块坐标，判断下一文本行是否有效即当前文本行

是否到达文档末尾，如果下一文本行有效，则提取并获取新文本行的坐标，指针指向新的文本行，比较新文本行是否满足文本块合并条件，如果满足，则扩大当前文本块边界使其包含新文本行，如果不满足，则存储当前文本块，并将新文本行坐标初始化为新的文本块，继续判断下一文本行是否有效，开始新的迭代。具体地算法如图 3.9：

算法 1 文本块生成

输入: pdf

输出: 文本块

```

1: function GETTEXTBLOCK(pdf)
2:   tempLine  $\leftarrow$  pdf.firstLine //前一文本行
3:   tempTextBlock  $\leftarrow$  tempLine //初始文本块
4:   for page1 to pageL do
5:     nextLine  $\leftarrow$  tempLine.nextLine //当前文本行
6:     if getCondition(tempLine,nextLine) then //满足条件，扩大当前文本块
7:       tempTextBlock  $\leftarrow$  tempTextBlock + nextLine
8:       result  $\leftarrow$  tempTextBlock
9:       tempLine  $\leftarrow$  nextLine
10:    else //不满足条件，保存当前文本块，并将当前行初始化为新的文本块
11:      result  $\leftarrow$  tempTextBlock
12:      tempTextBlock  $\leftarrow$  nextLine
13:      tempLine  $\leftarrow$  nextLine
14:    end if
15:  end for
16:  return result
17: end function
18:
19: function GETCONDITION(tempLine,nextLine)
20:   j  $\leftarrow$  middle
21:   k  $\leftarrow$  0
22:   result  $\leftarrow$  true
23:   if nextLine.x1 > tempLine.x2 then //处于不同栏
24:     result  $\leftarrow$  false
25:   else if nextLine.x2 < tempLine.x1 then //处于不同栏
26:     result  $\leftarrow$  false
27:   else if nextLine.y2 < tempLine.y1 and |nextLine.y2 - tempLine.y1| < 2 then //Y值满足规律
28:     result  $\leftarrow$  true
29:   else
30:     result  $\leftarrow$  false
31:   return result
32:

```

图 3.9 文本块生成算法

由于文本渲染的顺序不一定正确，因此，利用 PDFTron 按行读取文本，渲染顺序相邻两行的文本在页面布局上并不相邻，其坐标也不相邻，这往往出现在多栏的情况，由于布局有点复杂，因此其中包含的图像可能会占据整个页面宽度和其他特殊情况，因此需要在合并条件中考虑到多栏的情况，全局信息中已经保存了栏数值。

3.3 本章小结

本章节针对文本区域识别提供了解决方案，首先分析影响文本元素解析的因素，主要包含几个方面：从指定位置提取文本、PDF 呈现的不同、PDF 文本行提取、PDF 单词提取、以及渲染顺序。然后提出了基于文本坐标规律的识别方法，首先获取页面全局信息，然后解析 PDF 文件中的文本行，根据文本块间文本行的规律制定文本行合并条件，并设计了存储结构将生成的文本块进行存储。

第 4 章 非文本区域识别方法

本章节对影响 PDF 非文本区域解析效果的几方面问题进行了分析,将矢量元素还原并按一定的规则进行合并,提出了基于最大矩形区域扩充的非文本区域识别方法,并按设计的数据结构存储提取的非文本块,此解决方案适用于 PDF 中的矢量图、光栅图(带图片边框线条)、表格等矢量元素。

4.1 关键因素及影响分析

4.1.1 图形解析关键因素

PDF 中的图形内容是基于 PostScript 中使用的模型,它是由直线和曲线构成,可以对路径进行填充、描边,并且填充和以选用任何颜色或图案。它使用独立于设备的笛卡尔坐标系,通过矩阵来缩放、倾斜、放置图形元素来对 PDF 页面进行描述。对于 PDF 页面中的矢量图形,它由内容流构建,内容流包含一系列运算符及其后面附带的零个或多个操作数,PDF 中的图形运算符有 78 个^[30]。在 PDF 中,包含了 24 种图形状态属性,比较常用及重要的有以下几种:当前变换矩阵(CTM),它决定了坐标、剪切路径、色彩空间;通过依次使用运算符以及操作数来构建矢量图形,操作数通常是数字,但也可以是字典或数组。由于 PDF 采用独立于设备的用户空间坐标系,因此在对其进行解析时,需要将其坐标进行转换,使用的就是 CTM。矢量图形可以由多个子路径构成,并且其路径可以是直线与曲线,曲线由起点、终点和两个控制点组成,两个控制点定义了曲线在起点和终点之间的形状。对于表格而言,其线条为路径,通常为直线或矩形。对于光栅图像而言,通常光栅图像本身为嵌入图像,如果在生成 PDF 时对光栅图像有线条填充,则该光栅图像的线条元素为矢量图形。

对于 PDF 矢量图形即非文本区域的解析,我们需要考虑的问题主要为对其中的矢量元素进行合并以得到完整的矢量图形坐标,对于矢量图形及表格,需要将其中的矢量元素与文本进行合并,对于光栅图,需要将其中的矢量元素与光栅图像本身进行合并,需要考虑以下几个关键步骤:

- (1) PATH 绘制;
- (2) 坐标还原;
- (3) 矢量图形合成。

4.1.2 图形解析影响分析

在 PDF 在创建之初,图像便被拆开成了各个单独的部分,而不是作为一个整体来进行存储,并且不同的 PDF 创建工具对同一图像的存储方式会有不同。PDF

文件中包含两类图像，一类称为光栅图像，另一类为矢量图像。对于光栅图像，目前现有的工具提供了相关的接口，只是尚无法将其中线条一起获取，对于 PDF 中的矢量图像，2.3 节已经做出了具体介绍，矢量图像在存储时并不是作为一个整体，并且由于它不只包含单一的元素，因此需要针对其中不同的元素做出不同的处理，并对其合并，以识别出完整的矢量图像。PDF 中的矢量图形包含矢量形状、矢量图、表格、标线等，这类型内容在学术论文中相当常见，它是由矢量元素与文本元素相结合而成，因此通过元素类型遍历的方式并不能完整地提取它们，现有的 PDF 解析工具也没有提供有力的工具。

(1) 光栅图提取的困难：把从 PDF 中提取出的图像保存到另外的可编辑文件中，但是当 PDF 文件较大页数较多，或者需要对多篇 PDF 进行处理时，这种方法效率很低。

(2) 矢量图提取的困难：由于在 PDF 中矢量图像并不是存储为完整的图像文件，而是由 PATH 组合或 PATH 与文本组合而成，因此，要从 PDF 提取图像，需要从所有原始数据中组合图像，不能使用简单的对象提取方法来获取。

(3) 表格提取的困难：表格并不是被存储为整个的表格对象，而是采用与矢量图形同样的存储方式，由 PATH（表格框线）与文本组合而成，表格按照表格框线特征，分为既有水平线又有垂直线、只有水平线、包含不规则线的 3 种表格，表格线条提取较为困难，因为容易混入页面的其余线条元素。

4.2 基于最大矩形区域扩充的识别方法

我们的目的是对非文本区域进行识别并提取出非文本块，借助 PDFTron 提供的读取元素接口 `ElementReader`，根据元素的类型分别处理，对于元素类型为 `e_image` 的光栅图像，可以是使用 `ASCIHexDecode` 过滤器压缩的输入图像流、输入图像也可以经过 `Flate` 压缩、输入图像也可以是 `G3` 流、`G4` 流、`JP2` (`JPEG2000`) 图像、`JPEG` 图像、或者输入流未压缩。它一般是嵌入图像，可从缓冲区嵌入、从外部文件嵌入、从 `Image` 对象创建嵌入的图像、使用滤镜格式直接嵌入已压缩的图像。`com.pdftron.pdf.Image` 对图像的多种创建方式以及图像数据的提取提供了完整的接口，对于 `Image` 对象的创建，需要将 `Element` 的 `SDF / Cos` 字典（通过 `element.getXObject()` 获得）传递给 `Image` 构造函数，如获取当前图像的位图、图像色彩空间等，因此，对于光栅图像的提取，我们利用 PDFTron 对于 `e_image` 类型元素提供的接口提取，而光栅图像的边框线条在 PDF 解析中也被看成矢量元素 `e_PATH` 类型，对于元素类型为 `e_PATH` 的矢量元素，需要获取其 `PATHData` 的 `Point` 和 `Operator` 以及转换矩阵参数，通过 `PATH` 绘制和坐标转换将 `PATH` 还原。由于矢量图像以及表格往往包含有矢量元素以及文字，因此若将页面看成以正文

分割的各个区域,则要提取的非文本块应该最大地包含矢量元素以及非正文文本。
图 4.1 为具体的流程图:

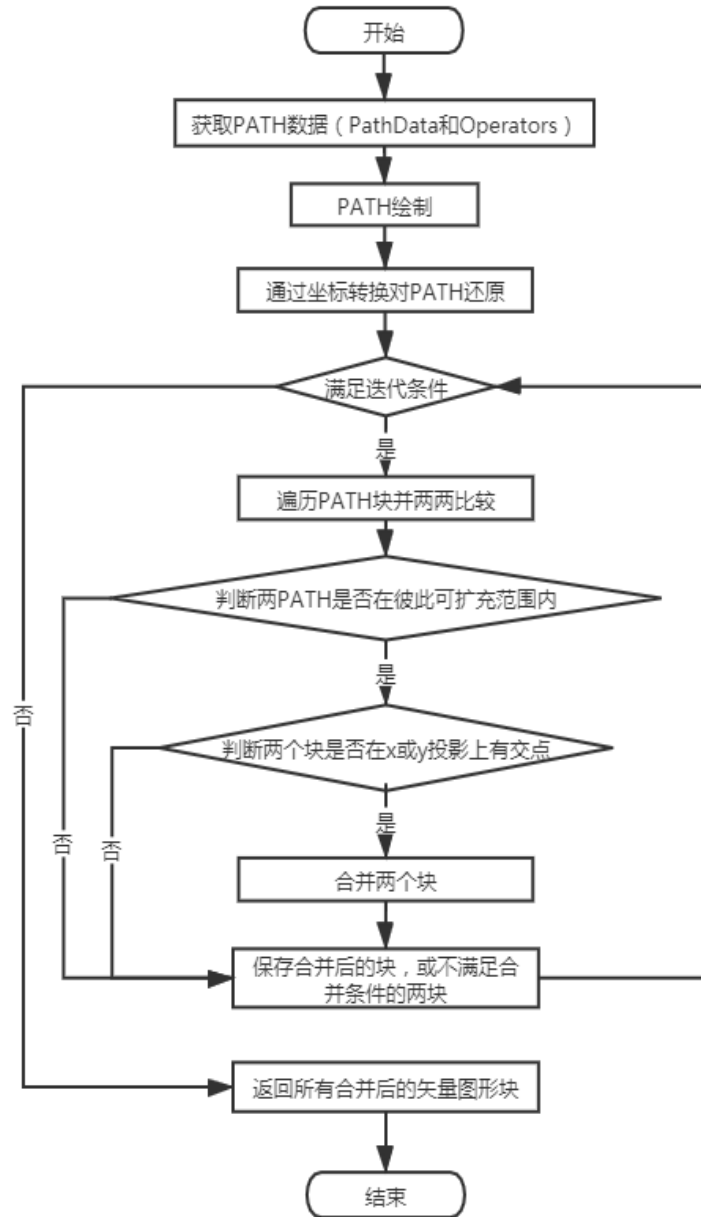


图 4.1 非文本区域识别流程图

同样地,对于解析出的非文本矢量图形块,我们设计了数据结构来进行存储,其数据结构类似 3.2 节介绍的文本块存储结构,所不同的是,对于图形,我们不需要包含过多样式,因此最里层的 map 保存的信息为坐标,因为根据坐标即可取出完整的区域。

4.2.1 PATH 路径重绘

在内容流中,PATH 对象基本上是点列表,点列表由一个起点和新加入的点组成,起点对应着指令 e_moveto,它意义着在给定的坐标处开始绘制路径,PATH

中的每个段可以是直线、曲线或矩形，分别对应指令 `e_lineto`、`e_cubicto`、`e_rect`，表示从路径中的上一个点到当前点绘制一条线段、一段曲线、一个矩形。因此 PATH 可以理解为描点和连线，在创建好 PATH 路径后，对 PATH 应用矩阵变换并将新的路径绘制在 canvas 画布上，通过 Stroke 属性设置线条填充，Fill 属性定义背景填充，填充可以是颜色或者图案。`getGState()`，`setPathStroke(true)`，`setFillColor`，`StrokeColor`。

因此在对 PATH 还原时，需要提取出 PATH 数据 PathData，包括其 Points 和 Operators，Point 对应着新加入的点，Operator 包含移动指令 `e_moveto`、画直线指令 `e_lineto`、画曲线命令 `e_cubicto`、画矩形指令 `e_rect`，通过不同的指令，连接新加入的点绘制出路径。如图 4.2 为重绘后但未经过变换的 PATH。

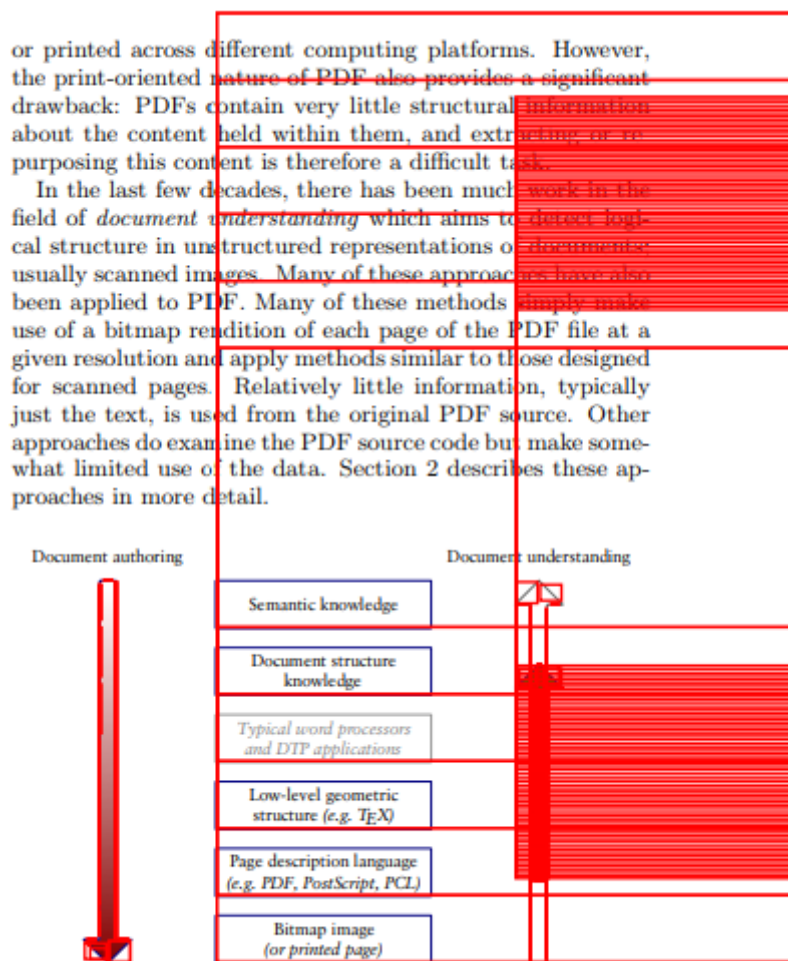


图 4.2 重绘后的 PATH

如图 4.2，我们发现重新绘制后的 PATH，形状还原了 PDF 中原图，但 PATH 中的点的坐标值并未进行转换，因此我们看到这些线段与矩形比实际的图片要大。

4.2.2 PATH 路径还原

PDF 坐标空间确定了 PDF 页面上 PDF 页面的布局以及对象的显示，在这个坐标空间上面包含了所有对象的大小、位置、方向等信息，而这些信息的表示采用的是笛卡尔坐标系，它的定义中包含三个因素：1.坐标原点位置。2.坐标系 X 轴、Y 轴的方向。3.坐标系每个轴的单位长度。在 PDF 中坐标系统分为设备坐标空间和用户坐标空间。

(1) 设备空间，每种用来显示 PDF 的设备都拥有自己独特的设备坐标系，使用此设备空间来做为 PDF 文档的坐标，可能会存在坐标原点不同、X 轴和 Y 轴的方向不同，因此在不同的设备上 PDF 上各对象的呈现会存在不同的差异。

(2) 用户空间：为了达到保证各种对象的相对关系能够在不同设备上保持一致，达到独立于设备的目标。PDF 定义了一种与设备无关的坐标系即用户空间，页面中各个部分对应着设备中的可成像矩形区域，这些矩形区域由页面 CropBox 定义，它代表了页面的裁剪区域。在用户空间中，原点或(0,0)点位于页面的左下角，X 轴方向向右延伸，Y 轴方向向上延伸。这个坐标系也称为默认用户坐标系，如图 4.3 所示。

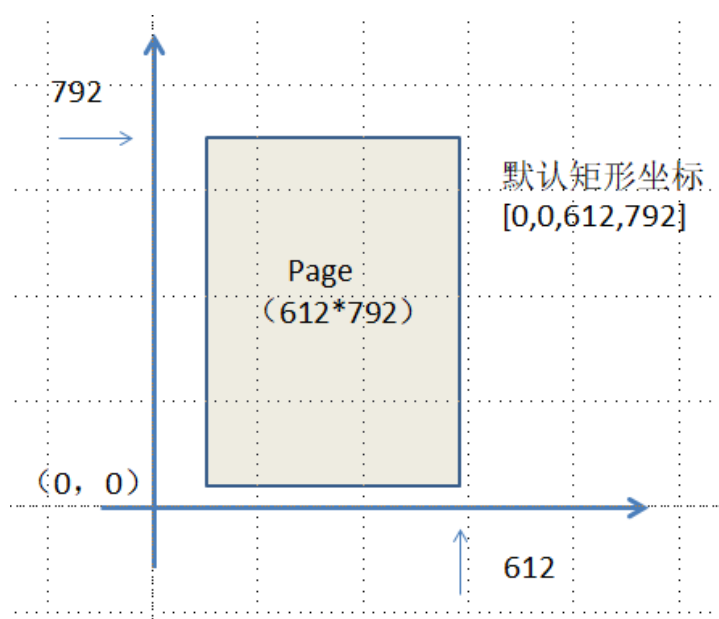


图 4.3 PDF 坐标系

PDF 页面有几个重要的矩形框，Bbox 决定页面的最外面边缘，而页面的绘图区域则由 MediaBox 定义，它有默认的矩形坐标[0,0,612,792]，坐标原点[0,0]位于页面的左下角，y 轴方向向上增大，x 轴方向向右增大。在设备空间坐标系与用户空间系的转换中，由转换矩阵（CTM）来定义两个坐标空间之间的关系，从而将用户空间坐标系转换至设备空间坐标。在 PDFTron 提供的工具中，Matrix2D 是 PDFTron 用来表示仿射变换的矩阵，它指定两个坐标空间的关系，通过它可以获

得仿射变换中矩阵的 6 个参数，A 代表第一行第二列的矩阵元素，B 代表第一行第二列的矩阵元素，C 代表第二行第一列的矩阵元素，D 代表第二行第二列中的矩阵元素，H 代表第三行第一列的矩阵元素，V 代表第三行第二列的矩阵元素，如图 4.4 Matrix2D 矩阵示意图。

$$\begin{bmatrix} A & B & 0 \\ C & D & 0 \\ H & V & 1 \end{bmatrix}$$

图 4.4 Matrix2D 矩阵

如前文所介绍的，仿射变换包括旋转、缩放、反射、剪切和平移^[51]，这些变换与参数对应的关系为：

- (1) 平移，由 dx,dy 指定，具体为 1, 0, 0, 1, dx, dy，对应矩阵如图
- (2) 缩放，由 sx,sy，若针对 (0,0) 为中心点，具体为 sx, 0, 0, sy, 0, 0，即对应矩阵
- (3) 旋转 (Rotating) 若以 (0,0) 为中心点以 x 弧度逆时针旋转，具体为 cosx, sinx, -sinx, cosx, 0, 0

要获取准确的位置信息，需要将当前矩阵与 CTM 连接起来，此步骤 PDFTron 提供了 multiply 方法，用于将当前矩阵乘以仿射矩阵得到结果矩阵，然后使用 multPoint 方法得到转换后的坐标，即将相对定位坐标与结果矩阵相乘，仿射矩阵与当前点相乘。如图 4.5 为经过变换还原后的 PATH。

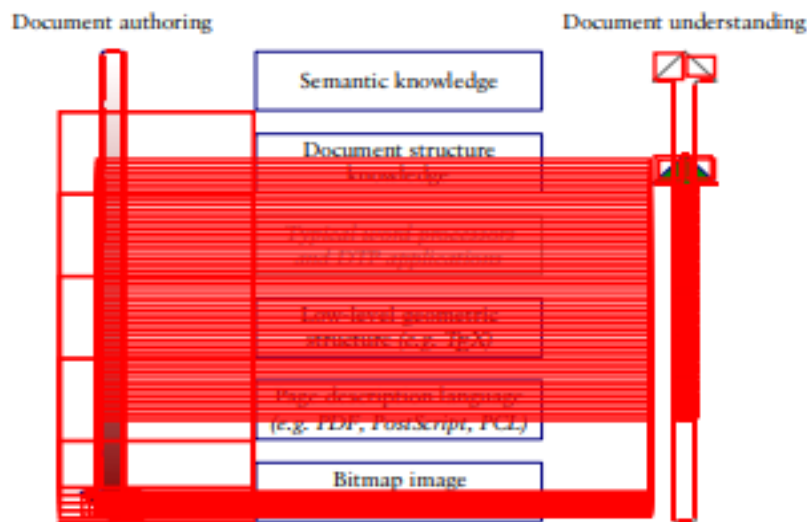


Figure 1: Document representation hierarchy

图 4.5 还原后的 PATH

4.2.3 矢量图形生成

经过 PATH 还原后，我们已经得到了矢量图形中的 PATH 数据，然而对于学术论文中常见的矢量图形如矢量图片、光栅图（有线条边框）、表格，这些图形一般由 PATH 与文本组合而成，因此通过元素类型遍历的方式并不能完整地提取它们，现有的 PDF 解析工具也没有提供有力的接口，这些图形包含的文本为非正文文本，其所占宽度不同于正文文本宽度，如果我们将页面看成是由正文文本块分割的各个区域，那么，一个完整的矢量图形的扩充边界只有两种情况，一：其与正文文本块的相交线，二：页面的边缘。因此我们提出了基于最大矩形区域扩充的非文本区域识别方法，具体的算法如图 4.6：

算法 1 矢量图形块生成

输入: listPath,listPara,hasHead

输出: 矢量图形块

```

1: function GETSVGBLOCK(pdf)
2:   if hasHeader then //如果有页眉,则跳过标线
3:     index ← 1
4:     listPath.remove(0)
5:   else
6:     index ← 0 //如果无页眉, 则从第一个矢量元素开始判断
7:   end if
8:   result.add(listPATH(index)) //result保存最终返回值
9:   for 0 to listPath.size do
10:    map ← listPath(index)
11:    flag ← true
12:    while flag do
13:      for 0 to result.size do
14:        base ← result[i]
15:        if InSideRange(base, map) and CombineCondition then
16:          //如果两个块互相在上下可延伸范围内, 且满足合并条件
17:          tmp ← (map + base) //tmp保存两块合并后的大块
18:          result.remove(base) //去重
19:          flag ← false
20:        else
21:          tmp ← map //不可以合并, 则tmp保存待加入的map
22:        end if
23:      end for
24:      if !flag then
25:        flag ← true //可以继续合并
26:      else
27:        tmp ← map
28:        break //不可以合并, 退出while
29:      end if
30:    end while
31:    result.add(tmp)
32:  end for
33:  return result
34: end function

```

图 4.6 矢量图形生成块生成算法

算法过程中遍历所有 PATH，由于页眉会影响 PDF 的 cropbox，从而导致矢量图形的扩充边界计算有误，并且页眉下方的标线也会被合并，从而导致合并后的矢量图形坐标区域不准确，因此首先需要判断 PDF 是否有页眉，如果有页眉，则

跳过标线，否则从第一个 PATH 开始判断，result 用来保存最终的返回值，两两比较 PATH 是否在其可延伸范围扩展边界内以及是否满足合并条件(相交或相切)，如果两个条件均满足，则将这两条 PATH 合并，并将合并后的大 PATH 块保存至 result,并参与后续的比较，由此 result 中保存了所有合并完全的矢量图形。

图 4.7 所示为表格的提取对比，图中上面图示中中图表被分别解析成了文字与 PATH 元素，文本与矢量元素被分别提取，因此要提取表格，需要对这些部分进行合并，同样可以看出，页面中的标线也会被解析成矢量元素，但它并不属于合并的范围，图 4.7 中下面图示为使用非文本区域识别方法提取表格效果。

WMT'14 English-German	BLEU		BLEU	Time (s)
Wu et al. (2016) GNMT	26.20	GNMT GPU (K80)	31.20	3,028
Wu et al. (2016) GNMT + RL	26.30	GNMT CPU 88 cores	31.20	1,322
ConvS2S	26.43	GNMT TPU	31.21	384
		ConvS2S GPU (K40) $b = 1$	33.45	327
		ConvS2S GPU (M40) $b = 1$	33.45	221
		ConvS2S GPU (GTX-1080ti) $b = 1$	33.45	142
		ConvS2S CPU 48 cores $b = 1$	33.45	142
		ConvS2S GPU (K40) $b = 5$	34.10	587
		ConvS2S CPU 48 cores $b = 5$	34.10	482
		ConvS2S GPU (M40) $b = 5$	34.10	406
		ConvS2S GPU (GTX-1080ti) $b = 5$	34.10	256

WMT'14 English-German	BLEU		BLEU	Time (s)
Wu et al. (2016) GNMT	26.20	GNMT GPU (K80)	31.20	3,028
Wu et al. (2016) GNMT + RL	26.30	GNMT CPU 88 cores	31.20	1,322
ConvS2S	26.43	GNMT TPU	31.21	384
		ConvS2S GPU (K40) $b = 1$	33.45	327
		ConvS2S GPU (M40) $b = 1$	33.45	221
		ConvS2S GPU (GTX-1080ti) $b = 1$	33.45	142
		ConvS2S CPU 48 cores $b = 1$	33.45	142
		ConvS2S GPU (K40) $b = 5$	34.10	587
		ConvS2S CPU 48 cores $b = 5$	34.10	482
		ConvS2S GPU (M40) $b = 5$	34.10	406
		ConvS2S GPU (GTX-1080ti) $b = 5$	34.10	256

图 4.7 表格提取

4.3 本章小结

本章节针对矢量元素识别提供了解决方案，PDF 中的矢量元素包括：矢量图形、表格、矢量图片以及光栅图片。首先对矢量元素解析的影响因素进行分析，然后提出了基于最大矩形区域扩充的识别方法，识别的第一步需要通过操作数及操作指令绘制 PATH 并进行坐标转换，然后将 PATH 与其周围的非正文文本按规则进行合并，以生成完整的矢量图形元素，并按照预先设计的存储结构将生成的矢量图形块进行存储。

第 5 章 块的合并与排序

本章节介绍了块的存储时需要用到的两个算法，合并算法主要针对于块间有重叠或交集，通过合并可以减少块间的冗余。排序算法主要是为解决页面元素的渲染顺序不一定严格按照从左至右、从上至下而带来的块的顺序不符合期望的问题。

5.1 块的合并算法

由于最初解析的文本块和 PATH 块之间存在重叠和交叉，如图 5.1 文本元素处理期间有很多块存在交集，这些交集位于 X 轴投影或 Y 轴投影上，通过合并和去除冗余，可以得到正确的段落块。如图 5.2 在画布上打印了 PATH，它们有很多重叠，我们可以在矩阵交换之前合并坐标，以减少 PATH 数量和存储时间。

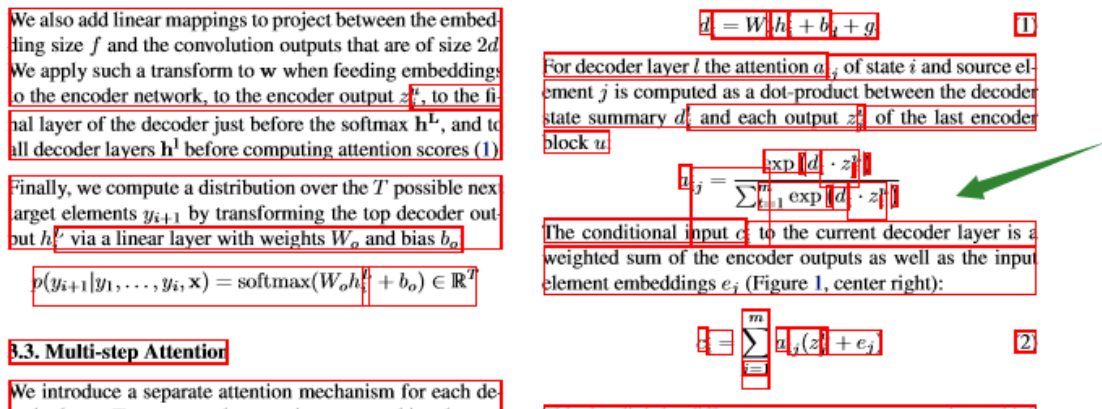


图 5.1 文本块存在交叉

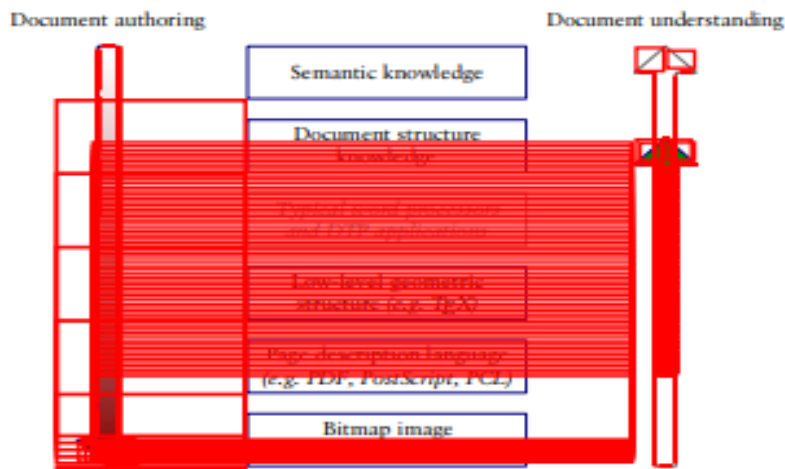


Figure 1: Document representation hierarchy

图 5.2 PATH 存在交叉

因此使用合并算法会将块的数量减少一个数量级，从而减少整个解析算法的内存消耗，具体过程如图 5.3：

算法 1 块的合并

输入: block1, block2

输出: result (合并后的块)

```

1: function IsCOMBINE(block1, block2)
2:   result  $\leftarrow$  block1, block2
3:   if IsCombine(block1, block2) then
4:     result  $\leftarrow$  block1 + block2 //result为block1与block2合并后的块
5:   else
6:     return result //result为之前未合并的块
7:   end if
8: end function
9: function IsCOMBINE(block1, block2)
10:  result  $\leftarrow$  false
11:  xState  $\leftarrow$  false
12:  yState  $\leftarrow$  false
13:  b1x  $\leftarrow$  block1.(x1 x2) //b1x,block1在x轴上的投影
14:  b2x  $\leftarrow$  block2.(x1 x2) //b2x,block2在x轴上的投影
15:  b1y  $\leftarrow$  block1.(y1 y2) //b1y,block1在y轴上的投影
16:  b2y  $\leftarrow$  block2.(y1 y2) //b2y,block2在y轴上的投影
17:  if block2.x in b1x OR block1.x in b2x then
18:    //一个块的x坐标包含于另一个块在x轴上的投影
19:    xState  $\leftarrow$  true
20:  end if
21:  if block2.y in b1y OR block1.y in b2y then
22:    //一个块的y坐标包含于另一个块在y轴上的投影
23:    yState  $\leftarrow$  true
24:  end if
25:  result  $\leftarrow$  xState & & yState
26:  return result
27: end function

```

图 5.3 块的合并算法

合并算法的过程是比较两个块（矩形）在 X 轴和 Y 轴上的垂直映射。如果它们在 X 轴上重合或相交，则在 X 轴上存在相交。如果它们在 Y 轴上重合或相交，则在 Y 轴上存在相交。只要在 X 轴上存在交点且在 Y 轴上也存在交点，就认为达到了合并条件。对于符合合并条件的两个块，将根据它们的坐标值确定合并，合并后的前两个块将被删除。

5.2 块的排序算法

使用排序算法是因为 PDF 内容流不一定有序，因此解析时会出现乱序的情况，并且在多栏的情况下，由于布局稍复杂，其中可能包含一个图片占据整个页面宽度等一些特殊场景，因此需要对解析结果中的各个块做排序。解释算法，排序算法的具体过程如图 5.4：

排序算法的过程中，需要考虑 PDF 学术论文的版面，两两比较块，如果全文

算法 1 块的排序

输入: block1, block2

输出: result(0: 不需要互换顺序, 1: 需要互换顺序)

```

1: function SORTBLOCK(block1, block2, column)
2:   if column == 1 then //一栏则按上下排列, 只考虑y
3:     if b1y < b2y then
4:       result ← 0
5:     else
6:       result ← 1
7:     end if
8:   else //多栏则按先从左至右, 再从上至下, 需要考虑x,y
9:     if (b1x < middle & b2x > middle) OR (b1x > middle & b2x < middle) then
10:      //两个块位于不同侧, 由x排序
11:      if b1x < b2x then
12:        result ← 0
13:      else
14:        result ← 1
15:      end if
16:    else //两个块位于同侧、或一个位于某一栏, 一个占整个页面宽(图), 由y排序
17:      if b1y < b2y then
18:        result ← 0
19:      else
20:        result ← 1
21:      end if
22:    end if
23:  end if
24:  return result
25: end function

```

图 5.4 块的排序算法

为一栏, 则排序时只需要考虑 Y 值, 如果全文为两栏, 则需要分情况考虑, 如果两个块分别位于不同侧, 则按 X 值排序, 位于左侧的块排在前面, 其余情况(两个块位于同侧、或一个块位于某一栏但另一个块占据整个页面宽度如图片)按 Y 值排序。

5.3 本章小结

本章节的两个算法用于文本块与矢量图形块存储时所需, 由于生成的块间会有重叠或交集, 为了避免块间的冗余而影响结果的准确性, 设计合并算法, 使得两个在 x 方向或 y 方向上有交集的矩形块合并成一个完整包含原先两个块的大矩形块。由于页面渲染顺序不一定严格按照从左至右、从上至下, 这种情况多出现在多栏的情况, 因此将导致识别生成的块的顺序也不符合期望, 排序算法分别针对页面排版为一栏和多栏的情况对两个相邻的块确定了顺序。

第 6 章 实验评估

本章节介绍了实验的设计与过程，包括实验数据集的构造、评估的标准。通过注释接口将算法结果与数据集进行比较，从文本、矢量图像以及表格三方面进行了实验测试，并与其它工具及算法进行了对比，实验结果以表格和图表的形式给出。

6.1 实验设计与过程

本实验的数据集构造，选取了不同期刊、不同领域、不同版面的 50 篇 PDF 学术论文，其中包含约 6218 个文本块、721 个矢量图形块、392 个表格。实验中设计了可视化的 web 平台，前端上传数据集中的 PDF 学术论文。

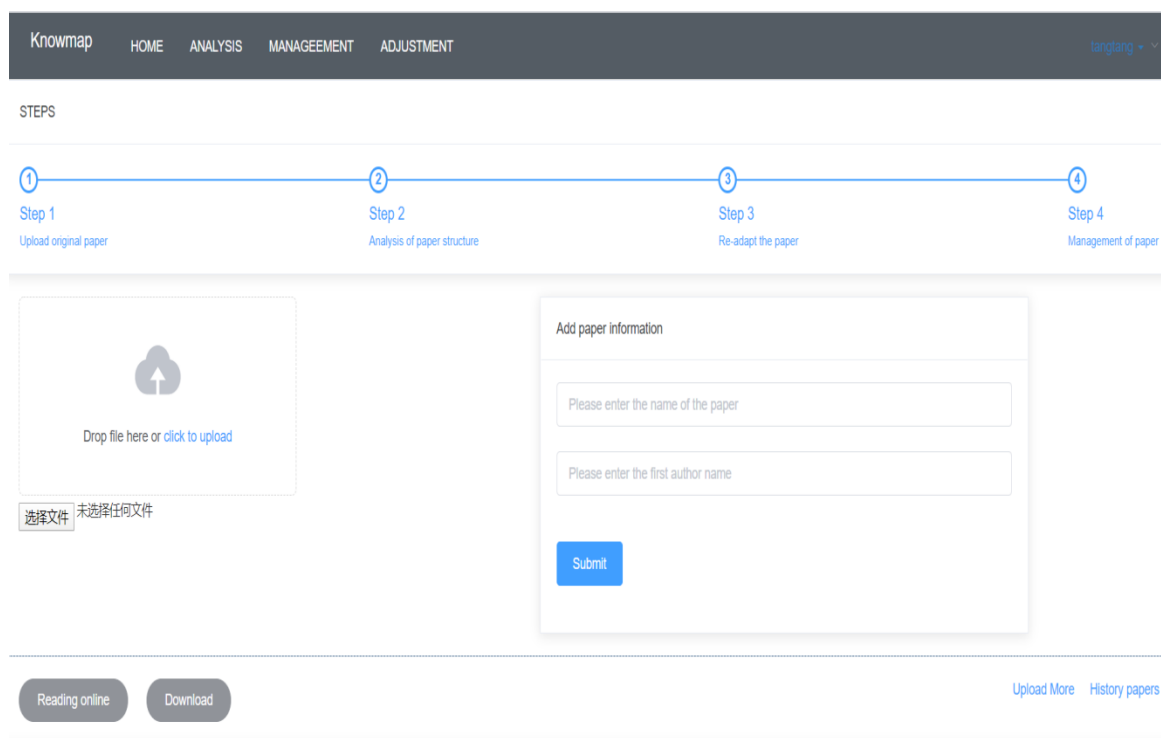


图 6.1 PDF 学术论文上传

如图 6.1 为学术论文上传页面，论文上传后会自动执行解析脚本对上传后的 PDF 学术论文进行解析，得到论文中块的相关信息并生成 JSON，可供查看及调整。

如图 6.2 所示为论文的管理页面，列表中为解析后的论文，可对其进行查看、调整框、下载以及删除。

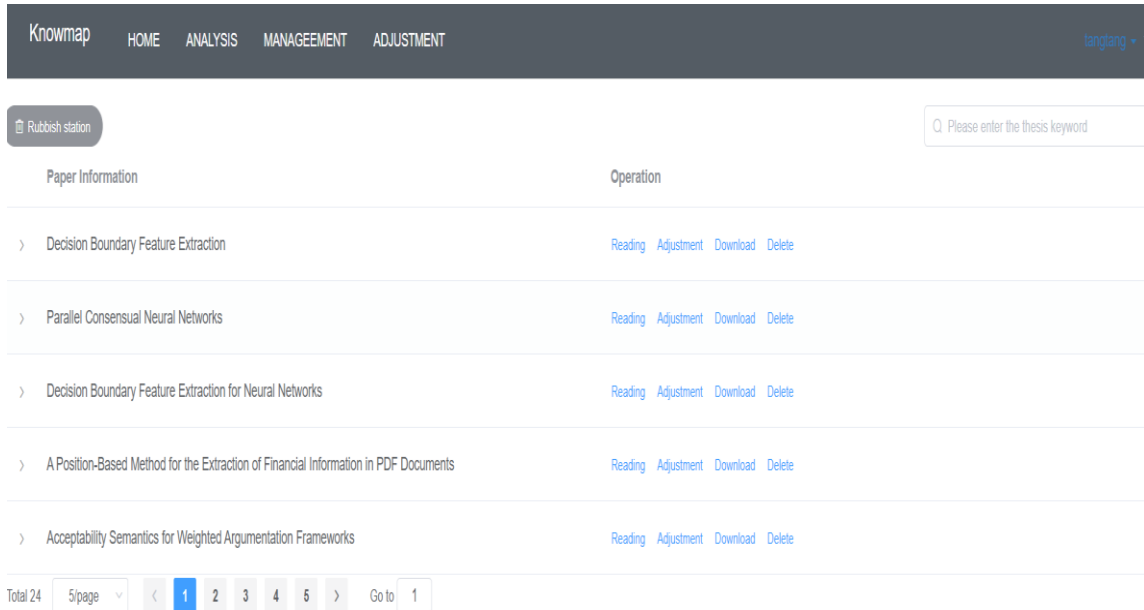


图 6.2 PDF 学术论文管理

根据 JSON 中的轮廓层次结构,提取出其中的类型及坐标信息,按顺序对 PDF 中的每个块生成注释框,如图 6.3。



图 6.3 PDF 学术论文自动化生成块信息

本实验采用自动化加人工校准的方式,利用解析算法生成的 JSON 文件,并通过人工调节的接口调节注释框,如图 6.4,提交后生成校准后的数据集 JSON 文件。

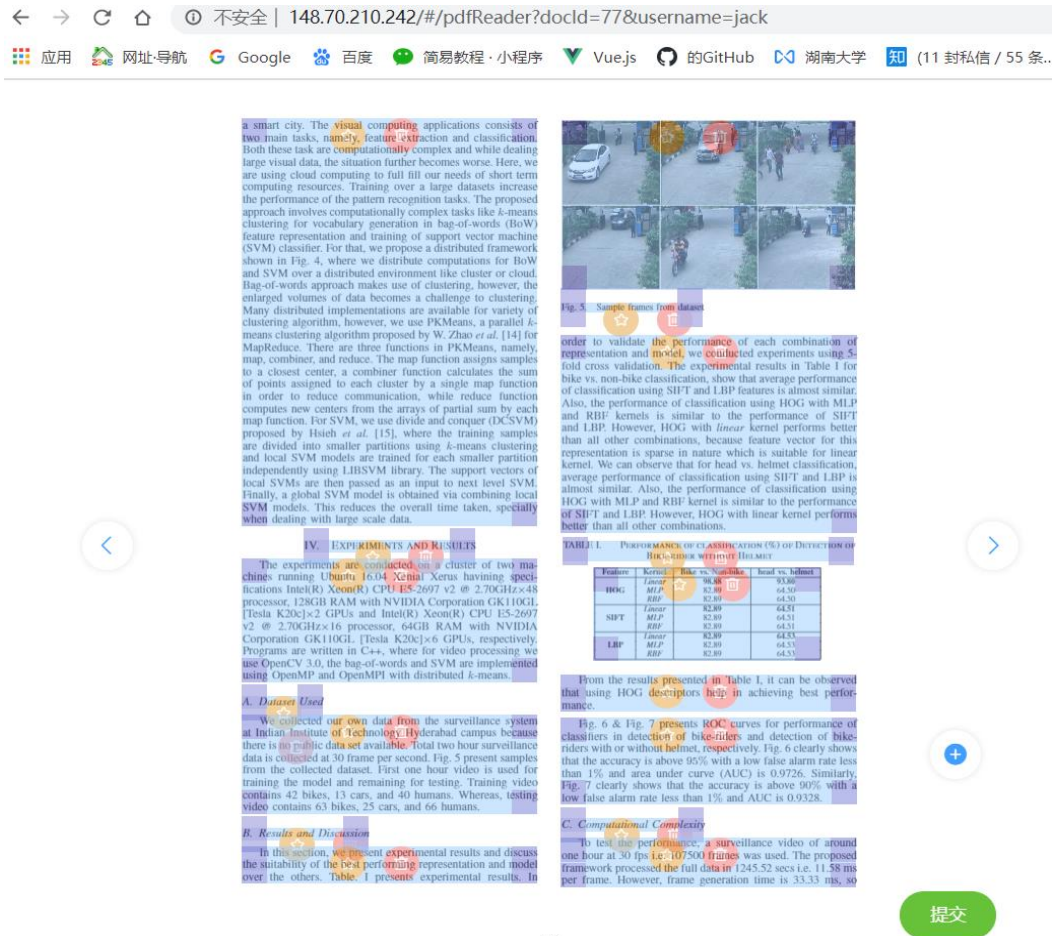


图 6.4 自动生成基础上人工调节块信息

本实验在 Windows 10 系统、IntelliJ IDEA 开发工具下,以 JAVA 开发语言实现学术论文信息提取过程,使用框架 PDFTron 辅助实现。为了更好地展示 PDF 解析过程,本文使用 2.5 节介绍的 PDFTron 提供的注释类 `com.pdftron.pdf.annots` 将识别的区域标记在页面中并设置边框样式。实验会提取出的块的类型及坐标信息保存至 JSON 文件中,JSON 富含了描述性标记,赋予了它们语义角色并反映其在 PDF 学术文件中的顺序及其轮廓层次结构,图 6.5 为 JSON 结构图示,JSON 中会保存页面的宽与高、块的集合,块中存储的信息有块的类型、块的坐标值。实验过程中,对通过两种识别方法所生成的块进行了合并,以缩小了块的数量级,并且设计了如 3.2 节所示的数据结构来保存所有块的信息,提供了许多快速查找的方法。

实验过程中我们发现,PDF 内容流的渲染顺序可能会因为页面存在多栏而出现乱序,因此在遍历元素时会先进行判断 PDF 是否有分栏并选择性的做跨栏处理,以重建顺序避免分块的失误,同时按照人的判断即从左至右从上至下设计了排序

算法，为提取出来的各个块进行排序。同时实验过程中我们发现矢量图形的处理比较麻烦，矢量图形往往是 PATH 组合或 PATH 与文本元素的组合，表格也是如此，因此我们不能单独通过提取文本元素或者矢量元素来获得，而应该采取一种合适的合并方式，为此，我们也设计了最大矩形区域扩充的识别算法。

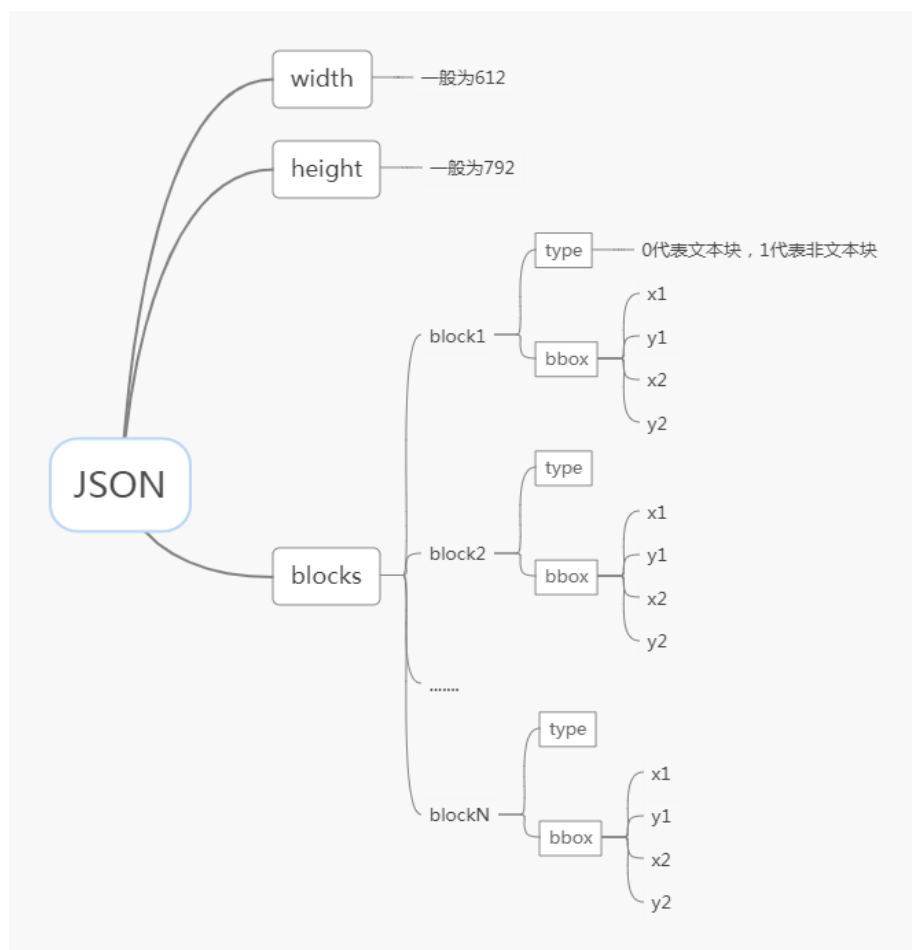


图 6.5 JSON 结构图示

实验的评估标准为块的类型正确与块的坐标位置小于阈值，阈值根据论文的字号来确定。实验结果统计过程为利用 2.5 节介绍的 PDFTron 提供的注释类 `com.pdftron.pdf.annots` 将识别所得的区域通过注释的形式标记在页面中，文本区域与非文本区域的注释框通过不同的背景颜色区分不同的类型，块的坐标信息通过矩形框标记出，与可视化平台中经自动化及人工校验后的数据集进行对比与记录，统计三部分实验内容的实验结果。第一：对文本块的提取；第二：对表格的提取；第三：对图像的提取。为了证实算法准确率，在实验过程中，将其与已有 PDF 解析库 PDFBox、PDFTron 以及文献 52、文献 45 中提出的算法进行了对比，实验结果表明本算法在非文本区域识别方面有良好的效果。

6.2 实验结果

图 6.6 为一篇双栏结构的学术论文用本文提出的基于文本坐标规律的识别方法以及基于最大矩形区域扩充的识别方法所得到的块的注释效果图示例，论文中包含文本以及矢量元素，图中块左边的注释框为绿色底代表块为矢量图形块，红色底代表文本块，注释框中的数字代表块的顺序。

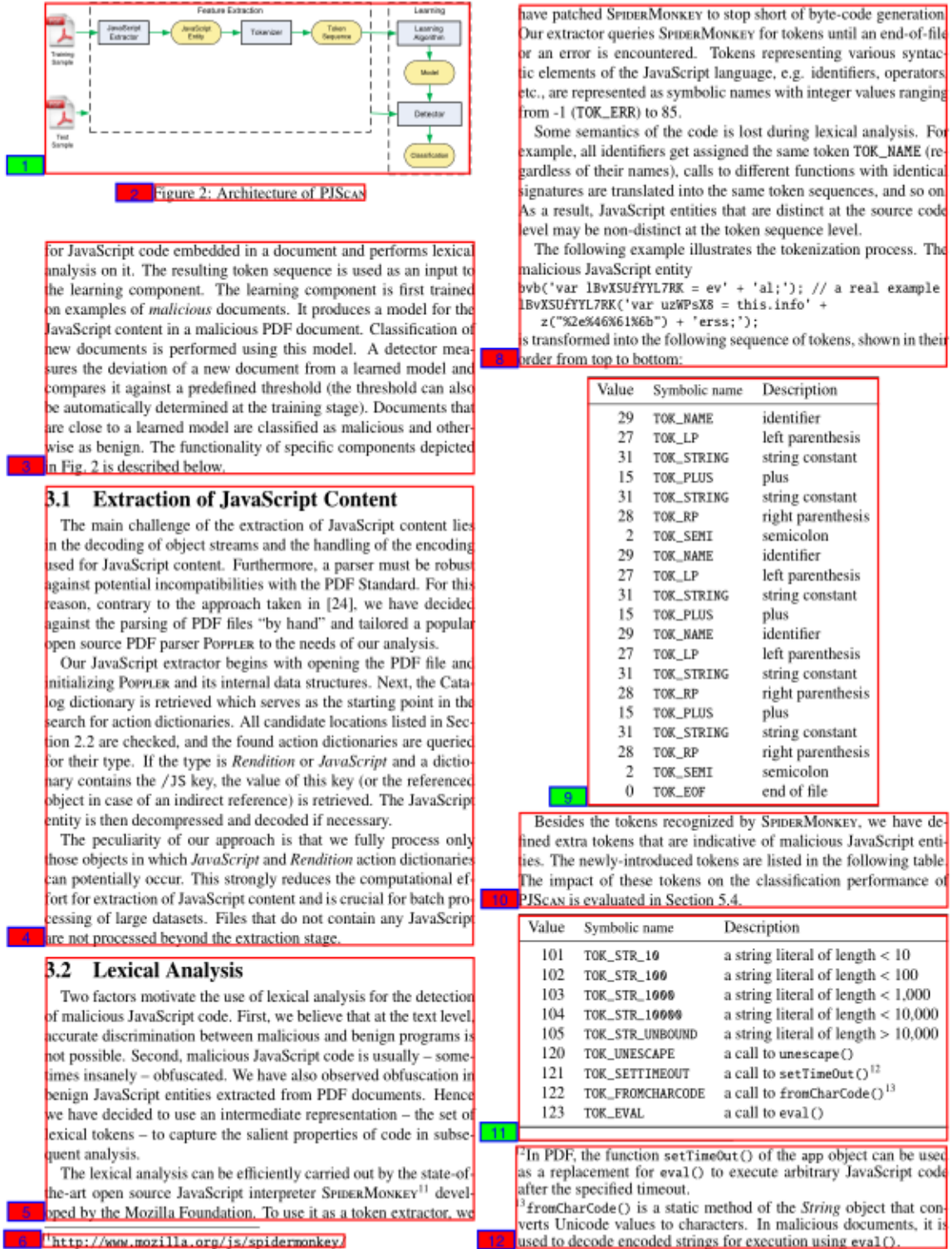


图 6.6 论文页面识别效果

6.2.1 文本

PDFBox 对于提取文本行有不错的效果，对于段落，它会把所有连着段落看成一个整体的段落，对于提取文本它存在的问题有：1.对于图形中的文本，会被识别成正文，如图 6.7 左边为原始 PDF 截图，右边为使用 PDFBox 提取出的文本，图中紫色矩形圈出来的区域本为图形的一部分，但却被识别成中文，因此生成的段落中会包含本不是正文的内容；2.文本的渲染顺序会因为矢量图而被打乱，如图 6.7 中绿色矩形圈出来的区域，它被放在矢量图中的文本之后提取，因此类似这样的情况，也会导致段落提取出现问题。具体如图 6.7。

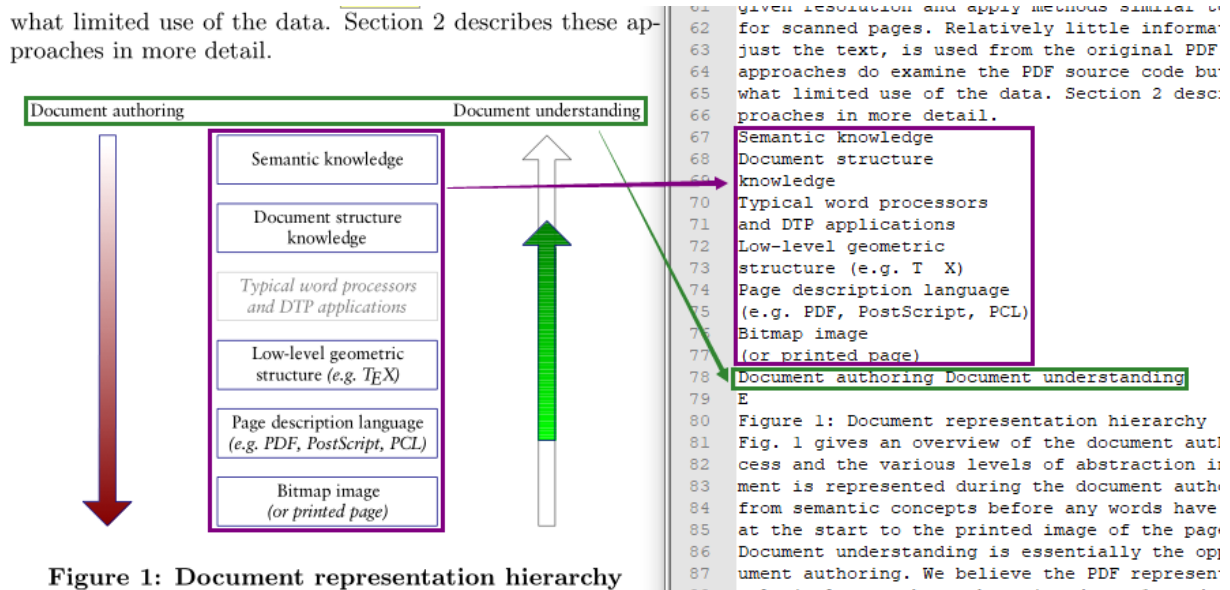


Figure 1: Document representation hierarchy

图 6.7 PDFBox 提取文本

文献 52 中基于 PDFBox 提出了对 PDF 文件的对象级文档分析的思想，提出“砖砌效应”连接文本片段垂直边缘来构建文本块^[52]。通常情况下，PDF 学术文档的排版都有具体的期刊及会议格式，而只有首行顶格输入的段落才会同时有左边和右边的垂直边缘，因此该方法无法对以缩进开头的段落进行处理，如图 6.8 为带边框的文字片段。

在该算法的评估中，对段落判定的正确表示采用了一种容忍的方法，允许以缩进开头的段落被合并，并且由于 PDF 数据集中包含大量带文本标签的图与表，而图与表中的文本会被识别为文本块，因此正确检测到的块集的比率没有预期的高，该算法中对文本块的测试结果如表 6.1：

表 6.1 对比算法中文本块识别统计

Item type	Total	True	False
Clusters	3157	2965(93.9)	192(6.1%)

本文所提出的文本区域识别方法基于 PDFTron，它与 PDFBox 存在同样的问题，并且 PDFTron 提供的获取文本行样式由行中第一个字符决定。因此，直接通过调用接口获取会得到误差。本文在 PDFTron 的基础上，扩展了全局信息类，并

通过调用低级 API 获取字符级别的信息，再利用投票机制得出行样式、页面样式等。然后通过文本行的坐标规律对文本行进行扩充以生成文本块，图 6.8 为文本行提取情况，图中绿色注释框中的数字为文本行的顺序。

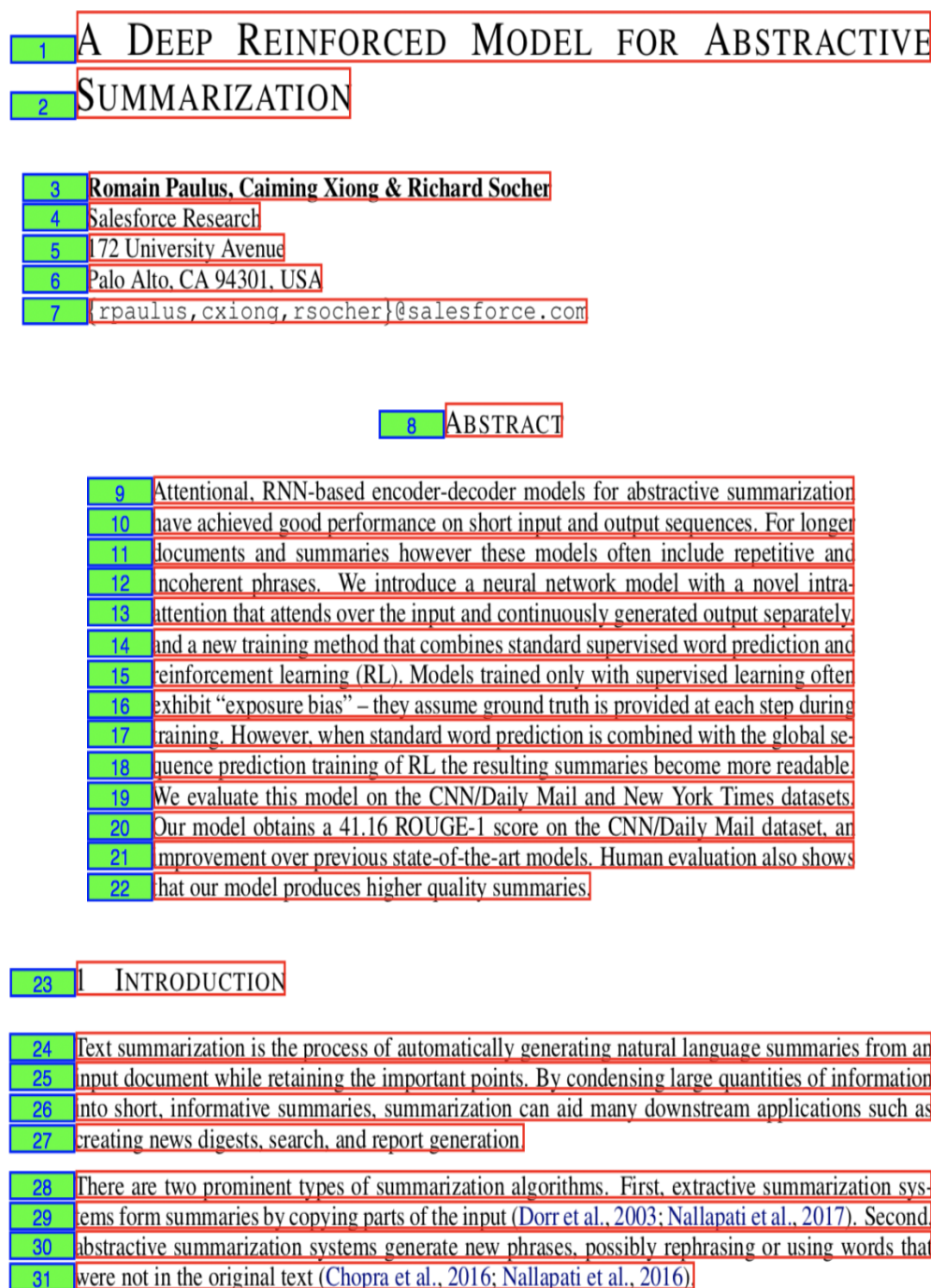


图 6.8 文本行提取

由于同一文本块间文本行的坐标值存在规则，处于同一文本块的相邻两行，当前行的 Y2 值小于等于上一行的 Y1 值，我们根据这一特征来制定文本块合并的规则。对文本行中满足合并条件的行进行合并，生成文本块。如图 6.9 为基于文本坐标规律的识别方法所提取的文本块。

实验中，对 50 篇 PDF 原始文档进行解析，记录文本块生成的实验结果，判断结果正确的评估标准为：类型正确。矩形框的坐标能最小地包含所有块内字符。对实验过程中产生错误的原因进行分析，主要有：1. 文本行提取会因为特殊字符

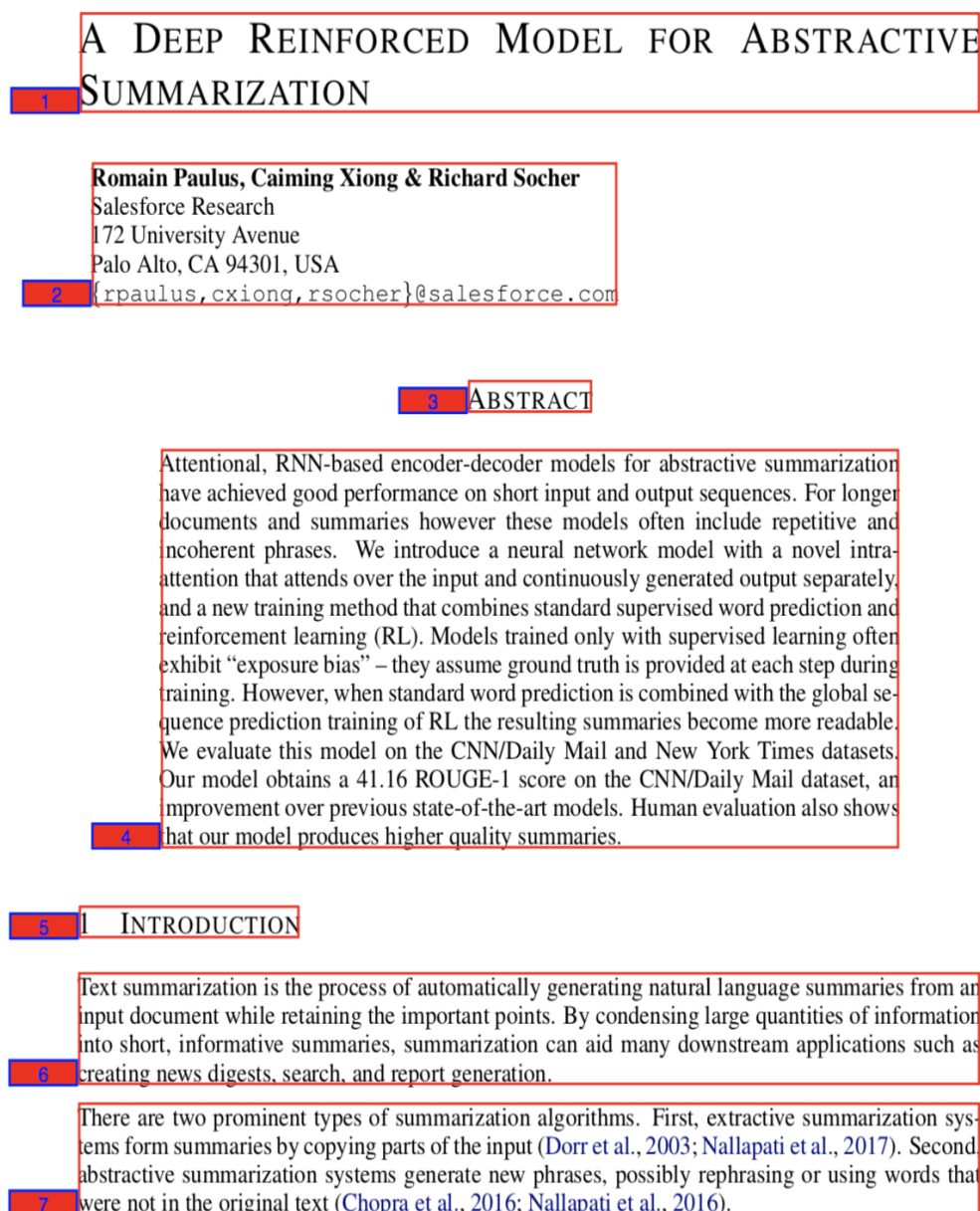


图 6.9 文本块坐标合并

而使文本行外接矩形框与相邻的文本行产生重叠，进而错误地将原本不位于同个一文本块的两段文本看成了位于同一文本块。2. 一些数学符号，如分号线，根号将会被识别成矢量元素，因此会影响文本块的识别效果。表 6.1 列出了文本块识别统计情况。

表 6.2 本算法中文本块识别统计

Element type	Total	Ture	False
textBlock	6218	5804 (93.3%)	414 (6.7%)

与对比算法中的实验数据对比，文本块识别统计数据的直方图如图 6.10。本算法在文本块总数为 6218 的情况下，识别的正确率为 93.3%，对比算法在文本块总数为 3157 的情况下，识别的正确率为 93.9%。对比算法的正确率比本文算法高出 0.6%，但本算法不局限于 PDF 文本的排版，不会受到段落首行缩进样式的影响，而对比算法中对缩进样式段落的合并采用了容忍的评估。并且在对比算法中，无法处理矢量图与表中的文本元素，它会将这类文本元素作为正文文本来处理，而实际上它们只是矢量图与表格的一部分，在 PDF 中，矢量图与表格并不被做为一个完整的 Object 来存储，而是由文本元素与矢量元素 PATH 组成，对于矢量图而言，PATH 有 line、rect、curve，而在表格中一般为 line 和 rect。而在本算法中，我们在处理非文本区域时，会将矢量元素与非正文文本进行合并以生成完整的矢量图，再将其与文本块进行去除，通过这种方法生成的文本块中将不再保留这类矢量图像及表格中的文本。

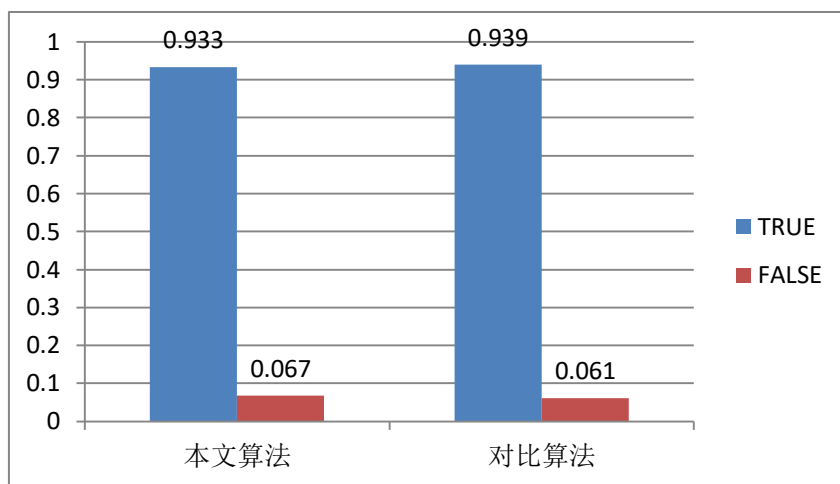


图 6.10 文本块识别对比直方图

6.2.2 图像

矢量图像由 PATH 与文本组成，利用 PDFBox 现有的接口只能识别光栅图像而无法识别到矢量图像，在图像提取过程中，会忽略掉所有的矢量图像，并且对于光栅图像，它的图片边线也无法识别。文献 52 基于 PDFBox 提出了一种 PDF 元素提取的简化模型^[52]，在该模型中，对于矢量图像的提取，用直线近似曲线，使用基于图的表示作为在交互中执行数据提取的基础，分割完成后，将创建另一个邻接图以将结果中的相信簇连接在一起，该邻接图用于表示文档并执行数据提取。对于 PDFBox 无法将图片的边线与图片识别成一个整体的问题，在该方法中没有得到很好地解决，图片边线会被识别成矢量图形中的直线，如图 6.11 为原始光栅图像，图 6.12 为文献 52 中 PDFBox 提取的图像。从图中可以看出，光栅图所包含的矢量元素（图像线条）没有与图像一起被提取。

针对页面中的部分图形元素中，文本并不包含在文本指令中，而是作为矢量图或位图的一部分的情况，该方法认为这类图形元素主要为广告，图形中的文本

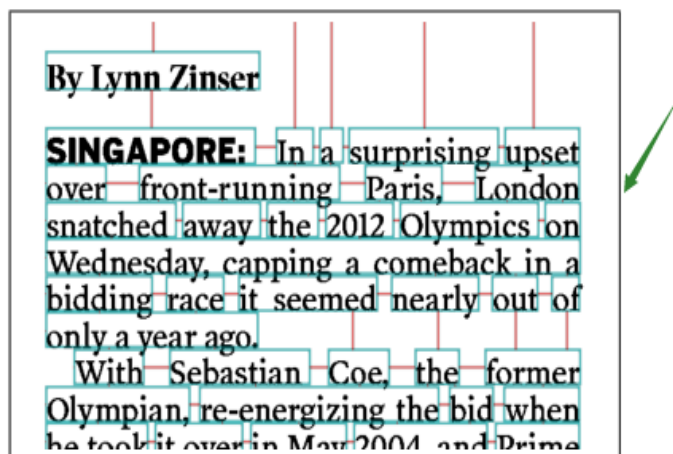
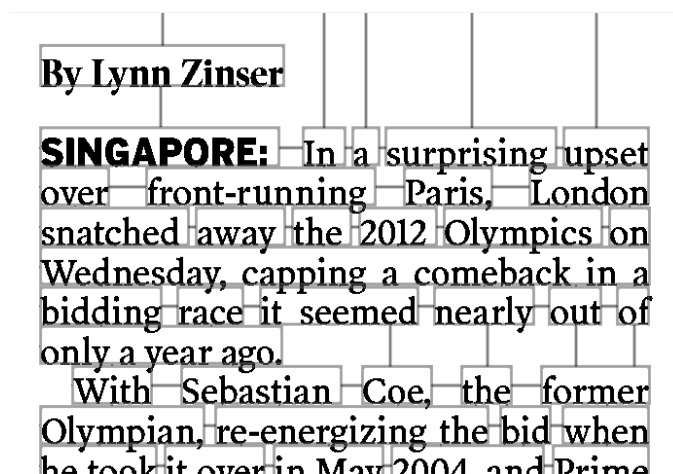
图 6.11 原始图像^[52]

图 6.12 PDFBox 提取的图像

并不是重要的元素，因此忽略这类图形元素，而实际过程中，矢量图占比很大。按照这一评估标准，该方法经测试后的结果如表 6.3：

表 6.3 对比算法中矢量图形块识别统计

Item type	Total	True	False
Rectangles	568	491(86.4%)	77(13.6%)

矢量图像由 PATH 与文本组成，利用 PDFTron 现有的接口只能识别光栅图像而无法识别到矢量图像，在图像提取过程中，会忽略掉所有的矢量图像。但 PDFTron 对矢量元素提供了低级的 API，可以获取到组成矢量元素的 PATH 数据，对于矢量图像的识别，我们的算法第一步是绘制它里面的 PATH，图 6.13.a 为 PATH 重绘的效果，第二步对 PATH 进行坐标转换还原，图 6.13.b 为坐标转换后的效果图。第三步对 PATH 进行仿射变换，图 6.13.c 为对 PATH 进行仿射变换后的效果图。第四步图 6.13.d 为对块进行合并去除冗余后的效果图。

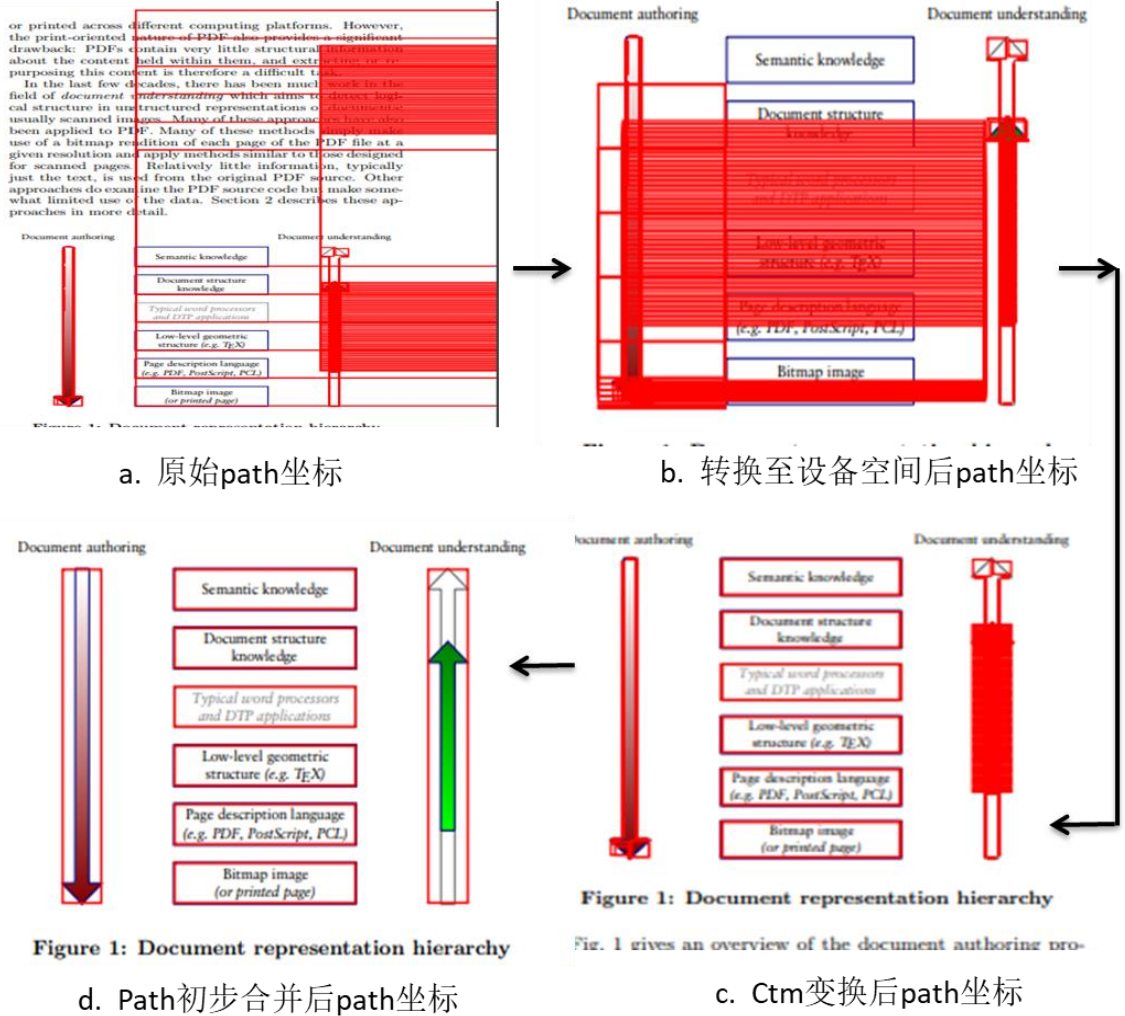


Figure 1: Document representation hierarchy

Fig. 1 gives an overview of the document authoring pro-

图 6.13 矢量图像识别

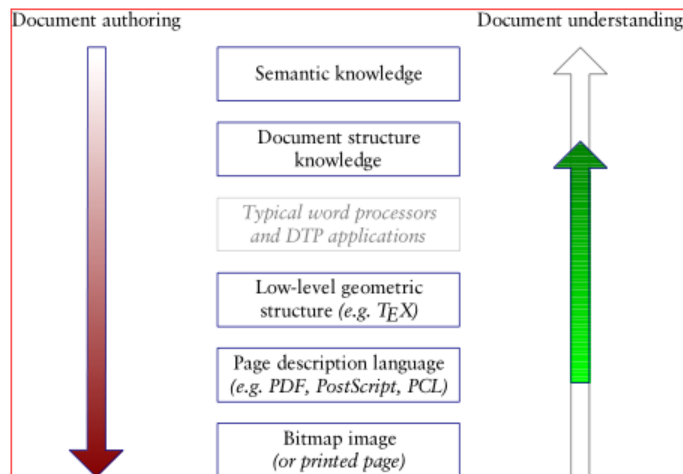


Figure 1: Document representation hierarchy

图 6.14 矢量图像最终坐标

将页面看成以正文分割的各个块，通过计算 PATH 块的最大可延伸范围，将在彼此可延伸范围内的 PATH 块与非正文文本合并，生成图 6.14 所示的最终识别的矢量图像。

实验过程中，产生错误的主要是页眉的判断不准，由于不同论文的排版不一样，因此页眉的特征也不一样，不能仅仅依靠其出现在同一位置，页眉中的文本内容一致这两个特征来判断，因此导致块的最大延伸区域计算有误，进而导致矢量图的识别不准确。表 6.4 为图像识别统计结果。

表 6.4 本算法中矢量图像块识别统计

Element type	Total	True	False
Img	721	656 (91%)	65 (9%)

与对比算法中的实验数据对比，矢量图形块识别统计数据的直方图如图 6.15。本算法在矢量图形块总数为 721 的情况下，识别的正确率为 90.9%，对比算法在矢量图块总数为 568 的情况下，识别的正确率为 86.4%。对比算法忽略了带文本的矢量图，并且不能很好地解决 PDFBox 中识别光栅图像而忽略掉图像边框线条的问题，本算法对比算法的正确率高 4.5%。

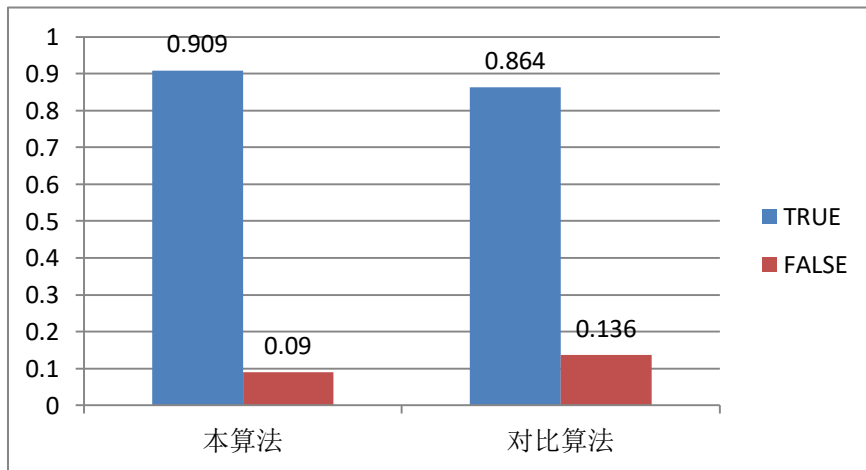


图 6.15 矢量图形块识别对比直方图

6.2.3 表格

PDF 中的表格并不是以整体来存储，在实验过程中，比较了现有的 PDF 解析工具，均未提供表格识别的接口，表格中的线条会作为矢量 PATH 被提取，文字元素也会被单独识别成文本，如图 6.16.a 为使用 PDFTron 解析表格并将文字被识别成正文的效果图，并且从图中可以看出，文本的渲染顺序不是按照从上至下，序号从 1 开始，当提取完第 6 行后转去解析表格外的顺序文本，重新开始解析表格时序号已经为 110。本文的算法中，对表格中的 PATH 即表格框线绘制并还原后，再将其与周围可扩展的非正文文本合并，生成的矢量图形效果如图 6.16.b。

			Operators
2	implemented by us	3	B, BI, c, CS, cs, Do, f, F, f*, h, K, k, l, m, n, q, Q, re, RG, rg, s, S,
		4	Tj, TJ, v, w, W, W*, y
5	Already implemented in PDFBox		BT, cm, d, ET, gs, T*, Tc, Td, TD, Tf, TL, Tm, Tr, Ts, Tw, Tz, \', \"
6	Not implemented	110	b, b*, B*, BDC, BMC, BX, d0, d1, DP, El, EMC, EX, G, g, i, ID, j, J, M,
		111	MP, ri, SC, sc, SCN, scn, sh

Figure 6: A list of operators which are implemented in our system

a. 表格中的文字被识别成正文



	Operators
Implemented by us	B, BI, c, CS, cs, Do, f, F, f*, h, K, k, l, m, n, q, Q, re, RG, rg, s, S, Tj, TJ, v, w, W, W*, y
Already implemented in PDFBox	BT, cm, d, ET, gs, T*, Tc, Td, TD, Tf, TL, Tm, Tr, Ts, Tw, Tz, \', \"
Not implemented	b, b*, B*, BDC, BMC, BX, d0, d1, DP, El, EMC, EX, G, g, i, ID, j, J, M, MP, ri, SC, sc, SCN, scn, sh

Figure 6: A list of operators which are implemented in our system

b. 矢量元素与文本合并后的矢量图形

图 6.16 表格识别

本文算法中, 表格的解析效果比较理想, 对表格中的 PATH 元素进行转换后, 求出每一个 PATH 元素最大矩形扩充区域, 将其与矩形扩充区域中所包含的非正文元素合并, 去除掉冗余后得到完整的表格, 实验中发生错误的可能为, 表格中存在这样的文本行, 它与正文栏宽相等且字体大小为正文文字的大小, 这种文本行被识别成了正文。因此, 在求 PATH 的最大矩形扩充范围时, 当遇到此行文本, 就停止扩充, 导致没有识别到完整的表格。表 6.5 为表格识别统计。

表 6.5 本算法中表格识别统计

Element type	Total	Detected	Undetected or False
Table	208	195 (93.7%)	13 (6.2%)

文献 53 提出了基于文本元素的 PDF 表格区域识别方法研究^[53], 该方法基于 pdf2xml 对 PDF 中的文本元素进行抽取, 将文本元素分类并将表格标题区域向下延伸, 从而识别表格区域。该方法的示例图如图 6.17。

该算法从 pubmed 的药学文献中, 选取了 50 篇构成数据集, 对表格区域识别的结果进行了统计, 如表 6.6 所示。

表 6.6 对比算法中表格识别统计

layout	Total	Detected	Undetected or False
Single column	58	52(89.7%)	6(10.3%)
Double column	63	59(93.7%)	4(6.3%)
Total	121	111(91.7%)	10(8.2%)

Table 5. ^{13}C and ^1H NMR Chemical Shifts for the Aglycon Moieties of Intercedenside G (4) and Intercedenside H (5) (in pyridine- d_5 /D $_2$ O, 4:1, 600/150 MHz)

position	δ_{C}	δ_{H} (J in Hz)	δ_{C}	δ_{H} (J in Hz)
1	35.9	1.33 (2H, m)	35.3	1.26 (1H, m, α)
2	26.7	1.80 (1H, m, β) 1.99 (1H, m, α)	26.5	1.34 (1H, m, β) 1.80 (1H, m, β) 1.98 (1H, m, α)
3	89.0	~ 1H (1H, m)	89.0	~ 2H (1H, m, β , α , β)
4	39.2		39.5	
5	47.6	0.91 (1H, dd, 4.2, 10.2)	47.9	0.93 (1H, m)
6	23.0	1.95 (2H, m)	23.0	1.91 (2H, m)
7	120.2	5.57 (1H, ba)	120.1	5.62 (1H, ba)
8	145.6		147.9	
9	47.2	3.33 (1H, d, 13.8)	47.5	3.37 (1H, d, 13.8)
10	35.3		35.6	
11	22.3	1.44 (1H, m, α) 1.76 (1H, m, β)	22.5	1.44 (1H, m, α) 1.72 (1H, m, β)
12	25.7	1.91 (1H, m) 2.67 (1H, m)	25.7	1.98 (1H, m) 2.67 (1H, m)
13	58.3		58.5	
14	47.9		48.3	
15	43.8	1.59 (1H, dd, β , 7.2, 12) 2.42 (1H, dd, α , 4.8, 8.4)	43.6	1.71 (1H, m, β) 2.51 (1H, dd, α , 4.8, 8.4)
16	72.7	5.91 (1H, m)	82.8	6.10 (1H, m)
17	57.2	~ 1H (1H, m)	57.2	
18	179.5		179.0	
19	23.7	0.90 (3H, s)	24.1	1.2 (3H, s)
20	84.0		85.9	
21	28.9	1.60 (3H, s)	27.0	1.79 (3H, s)
22	131.7	5.69 (1H, m)	128.6	5.77 (1H, d, 12)
23	120.3	6.03 (1H, m)	121.3	6.20 (1H, t, 12)
24	121.0	6.42 (1H, m)	122.5	6.69 (1H, d, 12)
25	~ 37.0		~ 37.0	
26	26.0	1.66 (3H, s)	26.2	1.60 (3H, s)
27	17.5	1.61 (3H, s)	17.7	1.57 (3H, s)
30	16.8	1.00 (3H, s)	17.0	1.08 (3H, s)
31	28.2	1.16 (3H, s)	28.3	1.15 (3H, s)
32	32.4	1.08 (3H, s)	31.5	1.49 (3H, s)
CH ₂ COO	170.2		171.1	
CH ₂ COO	21.2	1.92 (3H, s)	21.1	1.94 (3H, s)

↓ 延伸位置1
↓ 延伸位置2
↓ 延伸位置3
↓ 延伸位置4
↓ 最终延伸位置

$\text{C}_{55}\text{H}_{85}\text{O}_{26}\text{SNa}$ from pseudomolecular ion peaks at m/z 1237 $[\text{M} + \text{Na}]^+$ in positive-ion mode ESIMS and at m/z 1191 $[\text{M} - \text{Na}]^-$ in negative-ion mode ESIMS. A fragment ion peak at m/z 1117 $[\text{M} - \text{OSO}_2\text{Na} - \text{H} + \text{Na}]^+$ in the positive-ion mode ESIMS indicated the presence of a sulfate group in the glycoside. The IR spectrum showed the presence of hydroxyl (3437 cm^{-1}), carbonyl (1732 cm^{-1}), olefinic (1671 cm^{-1}), and sulfate ($1245, 1069\text{ cm}^{-1}$) groups.

On the basis of HMQC, TOCSY, DQF-COSY, and HMBC spectra, all ^1H and ^{13}C signals were assigned as shown in Table 5 and Table 6. A NMR spectral comparison of 5 with

The site of the sulfate linkage was determined by comparing the ^{13}C NMR data of compound 5 with those of known glycosides.⁷ A downfield esterification shift was observed for the C-4' signal (xyl, from δ 68.2 to 75.8 ppm). Therefore, the structure of compound 5 was deduced as 16 β -acetoxy-3-*O*-(3'-*O*-methyl- β -D-glucopyranosyl(1 $\rightarrow$ 3)- β -D-xylopyranosyl(1 $\rightarrow$ 4)- β -D-quinovopyranosyl(1 $\rightarrow$ 2)-4'-*O*-sulfate- β -D-xylopyranosyl) holosta-7,22Z,24-triene-3 β ,17 α -diol sodium salt.

Intercedenside I (6) was obtained as a colorless amorphous powder. The molecular formula of intercedenside I

图 6.17 文献 53 中表格延伸示意图^[53]

与对比算法中的实验数据对比，表格识别统计数据的直方图如图 6.18。本算法在表格总数 208 的情况下，识别的正确率为 93.7%，对比算法在表格总数为 111 的情况下，识别的正确率为 91.7%。对比算法在识别过程中存在将非表格文本元素与表格过度合并的情况，并且单栏的表格识别略差。在本算法中，存在的问题是，当表格中的文本行的宽度、字体样式与正文文本行一致时，会导致块的最大矩形扩充范围计算有误。因此本算法对于正文的判断应该补充更多的规则，以与表格中符合目前正文判断规则的文本行作出区分。

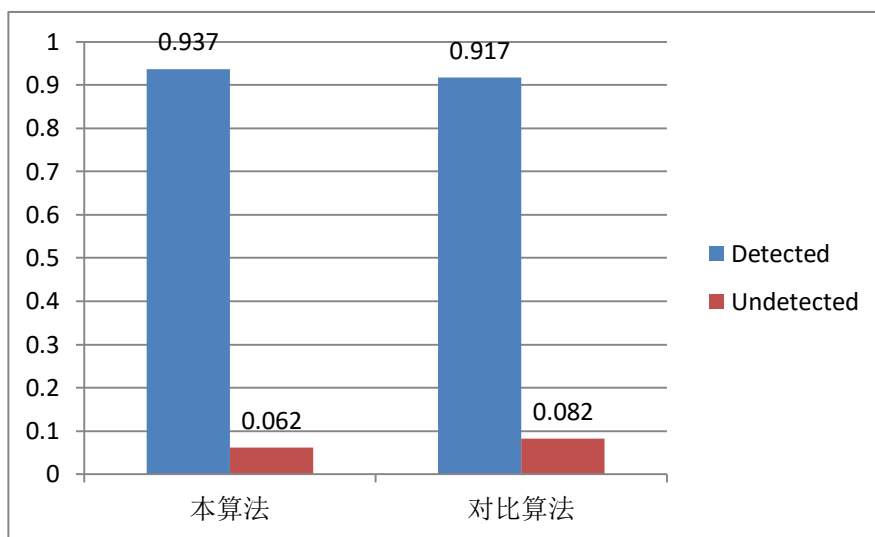


图 6.18 表格块识别对比直方图

6.3 本章小结

本章节对实验的设计与过程进行了介绍，以不同领域、具有不同排版期刊的 50 篇论文为测试论文集，分别对文本块、矢量图像、表格进行了识别，用 PDFTron 提供的注释接口将其坐标打印以查看识别效果，并将本算法与现有的工具 PDFBox、辅助实现的原始 PDFTron 以及现有算法进行了对比，并用表格及图表展示了测试的统计情况。最后对实验中出现的错误进行了分析。

结论

1.工作总结

本研究所提取的结构信息，可用于很多学术论文应用，目前有一些通过提取文本内容来实现 PDF 文件基于语义信息的查询，而且还能用于提取文件中的一些特定信息，例如，获得论文的标题、作者等相关数据信息。如通过提取摘要，作为文献综述生成器的原始数据。如通过提取的页面元素，来针对各个终端进行适配。因此对 PDF 学术论文结构内容的提取是一项即有意义又有挑战的研究。本文从文本元素和矢量元素两方面介绍了解析的关键问题，对影响其解析结果的因素进行了分析并提出了解决方案，针对文本区域提出了基于文本坐标规律的识别方法，将文本块内的文本行的坐标规律作为文本块扩充的条件；针对非文本区域提出基于最大矩形区域扩充的识别方法，将正文看成以页面分割的各个区域，对提取的矢量元素按左右下方向扩充，以最大范围地包含矢量元素及非正文文本，识别出非文本块。考虑到识别出的块与块之间的冗余与重叠，设计了块的合并算法，并对于多栏版面的 PDF 渲染顺序不正确引起的生成的块顺序不正确的问题设计了排序算法。工作总结按点展开如下：

(1) 通过整理相关研究文献以及对现有工具调研，对现有的 PDF 解析方法做了文献综述，并归纳了相关方法所带来的局限性，指出了 PDF 解析的必要性。

(2) 介绍 PDF 如何绘制文本对象，并对文本对象解析的关键因素以及影响其解析效果的原因进行了分析，通过文本对象坐标位置的特点，提出了基于文本坐标规律的识别方法。

(3) 介绍了 PDF 中图形对象的绘制，对矢量图形解析的关键因素以及影响其解析效果的原因进行了分析，提出了基于最大矩形区域扩充的识别方法，通过对 PATH 对象的重绘与还原，并将其与矢量图形中的文本元素进行合并，以实现非文本区域的识别。

(4) 为减少整个解析算法的内存消耗，提出了块的合并算法，以解决由于最初解析的文本块和 PATH 块之间存在重叠和交叉带来的冗余问题。

(5) 为使得识别的块的顺序不符合期望，提出了块的排序算法，以解决由于文本行的乱序而引起的块的乱序问题。

(6) 采用自动化加人工校准的方法，构造了适用于 PDF 学术论文的数据集。

实验结果表明，本文中提出的识别算法能很好地提取 PDF 结构信息，能够自动化地对学术资源进行抽取与处理，有利于 PDF 在学术论文领域中的进一步利用，对当前学术论文资源知识挖掘研究具有重要意义。

2.展望

由于 PDF 学术论文排版的多样性,使得对它的解析变得复杂,增加了工作的难度,对于学术论文而言,论文的作者也可能存在粗心,比如段首没有缩进,或者段内相邻两行字体大小发生变化,则会影响分段,这给解析工作带来了很大的难题,我们的算法只适用于正常排版的论文,毕竟这是大多数。对于论文中的公式,我们的解析方法尚存在不足,这是因为划线会被识别成 PATH,而公式中的其它文字会被当成文本,但公式中的特殊符号其样式往往不同于正文文本,它所占的坐标区域往往比实际区域大,这使得我们无法将公式单独提取出来,但由于它属于段落中的一部分,而我们的合并算法会因为它与段落有交集将它们进行合并去除冗余,具体如下:

(1) 对于文本块中的一些数学符号,如分号线,根号,会被识别成 PATH,从而影响文本块的合并。

(2) 对于有些含有页眉的 PDF 文档,页面下方的标线会被识别成 PATH,如果此标线下方为非文本元素,在矢量图形块生成时它也会被合并,从而影响非文本区域识别。

(3) 对于有些表格,它里面存在这样的文本行与正文栏宽相等且字体大小为正文文字的大小,这种文本行被识别成了正文,因此,在求 PATH 的最大矩形扩充范围时,当遇到此行文本,就停止扩充,导致没有识别到完整的表格,因此对于正文的判断应该补充更多的规则,以与表格中符合目前正文判断规则的文本行作出区分。

(4) 目前算法无法单独提取出 PDF 学术论文中的行间与行内公式。

本文提出的基于文本坐标规律的识别方法以及基于最大矩形区域扩充的识别方法还存在一些不足之处,由于 PDF 学术论文版面没有统一的标准,也存在很多人为干扰因素,对于文本块的提取,会因为特殊字符的原因导致文本块过度合并,同学对于文本块的中某些特殊数学符号,如分号线、根号线会被识别成矢量元素,因此影响文本块的识别效果。对于图像的识别而言,若创建者在页面的起始位置不正确地留有大片空白则会导致页面起始位置的计算有误,从而导致块的最大延伸区域计算有误,进而导致图像的识别不准确。在表格的识别中也会因为创建者对表格字体及正文字体未做出区分,从而导致表格中字体被识别成正文而影响表格的识别。因此在未来研究工作中会对相关提取的规则进行完善,以提高 PDF 结构解析的精确性。目前对于公式的处理,主要的难点在于特殊字符的处理以及找到合适的合并规则,对于行内的公式,由于它们被包含在文本块中,可以与文本块一起识别,而对于行间的公式,它们被作为文字存储,夹杂 PATH 类型的特殊符号,并且对于其行的提取毫无规律可言,因此对于行间公式的提取成了本文未来研究工作的一个重点。在未来研究工作中也会对相关提取的规则进行完善,以提高 PDF 结构解析的精确性。

参考文献

- [1] Adobe Systems Incorporated. PDF Reference, sixth edition. https://www.adobe.com/content/dam/acom/en/devnet/pdf/pdf_reference_archive/pdf_reference_1-7.pdf,2006-11.
- [2] Adobe Systems Incorporated. PostScript Language Reference, third edition. <https://www.adobe.com/content/dam/acom/en/devnet/actionscript/articles/PLRM.pdf>,1999-2.
- [3] 孙殷, 王鹏. PostScript 文件与 PDF 文件间数据转换. 微型机与应用, 2013, 32(11):19-21.
- [4] Wikipedia. PDF. <https://en.wikipedia.org/wiki/PDF>.
- [5] Wang Z, Beyette D, Lin J, et al. Extraction of Math Expressions from PDF Documents based on Unsupervised Modeling of Fonts. In: Proceedings of the International Conference on Document Analysis and Recognition (ICDAR). IEEE, 2019: 381-386.
- [6] Tensmeyer C, Morariu V I, Price B, et al. Deep Splitting and Merging for Table Structure Decomposition. In: Proceedings of the International Conference on Document Analysis and Recognition (ICDAR). IEEE, 2019: 114-121.
- [7] Morris D, Tang P, Ewerth R. A neural approach for text extraction from scholarly figures. In: Proceedings of the International Conference on Document Analysis and Recognition (ICDAR). IEEE, 2019: 1438-1443.
- [8] Bansal A, Harit G, Roy S D. Table extraction from document images using fixed point model. In: Proceedings of the 2014 Indian Conference on Computer Vision Graphics and Image Processing. 2014: 1-8.
- [9] Gao L, Yi X, Liao Y, et al. A deep learning-based formula detection method for PDF documents. In: Proceedings of the 14th IAPR International Conference on Document Analysis and Recognition (ICDAR). IEEE, 2017, 1: 553-558.

- [10] 任林涛. PDF 格式中文科技论文的有效信息提取方法及分类研究: [吉林大学硕士学位论文]. 长春: 吉林大学, 2011.
- [11] Boukhers Z, Ambhore S, Staab S. An end-to-end approach for extracting and segment-ing high-variance references from PDF documents. In: Proceedings of the ACM/IEEE Joint Conference on Digital Libraries (JCDL). IEEE, 2019: 186-195.
- [12] Beyette D, Wang Z, Lin J, et al. Semi-Automatic LaTeX-Based Labeling of Mathematical Objects in PDF Documents: MOP Data Set. In: Proceedings of the ACM Symposium on Document Engineering 2019. 2019: 1-4.
- [13] Li P, Jiang X, Shatkay H. Extracting Figures and Captions from Scientific Publications. In: Proceedings of the 27th ACM International Conference on Information and Knowledge Management. 2018: 1595-1598.
- [14] Potvin B, Villemaire R, Le N T. A Position-Based Method for the Extraction of Financial Information in PDF Documents. In: Proceedings of the 21st Australasian Document Computing Symposium. 2016: 9-16.
- [15] Shigarov A, Mikhailov A, Altaev A. Configurable table structure recognition in un-tagged PDF documents. In: Proceedings of the 2016 ACM Symposium on Document Engineering. 2016: 119-122.
- [16] Beel J, Langer S, Genzmehr M, et al. Docear's PDF inspector: title extraction from PDF files. In: Proceedings of the 13th ACM/IEEE-CS joint conference on Digital libraries. 2013: 443-444.
- [17] Chao H. Graphics extraction in a PDF document. In: Proceedings of the Document Recognition and Retrieval X. International Society for Optics and Photonics, 2003, 5010: 317-325.
- [18] Yu W, Li Z, Zeng Q, et al. Tablepedia: Automating pdf table reading in an experimental evidence exploration and analytic system. In: Proceedings of The World Wide Web Conference. 2019: 3615-3619.
- [19] Hoffswell J, Liu Z. Interactive Repair of Tables Extracted from PDF Documents on Mobile Devices. In: Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems. 2019: 1-13.
- [20] Wang X, Liu J C. A Font Setting Based Bayesian Model to Extract Mathematical Expression in PDF Files. In: Proceedings of the 14th IAPR Intern-

- ational Conference on Document Analysis and Recognition (ICDAR). IEEE, 2017, 1: 759-764.
- [21] Rastan R, Paik H Y, Shepherd J, et al. TEXUS: table extraction system for PDF documents. In: Proceedings of the Australasian Database Conference. Springer, Cham, 2018: 345-349
- [22] Perez-Arriaga M O, Estrada T, Abad-Mota S. TAO: system for table detection and extraction from PDF documents. In: Proceedings of the Twenty-Ninth International Flairs Conference. 2016.
- [23] 宋艳娟. 基于 XML 的 HTML 和 PDF 信息抽取技术的研究: [福州大学硕士学位论文].福州: 福州大学,2006.
- [24] 刘平,廖兆存,于俊清.科技文档对象在标签 PDF 中的表示研究.计算机工程与设计,2007(13):3198-3200.
- [25] Rahman F, Alam H. Conversion of PDF documents into HTML: a case study of document image analysis. In: Proceedings the Thrity-Seventh Asilomar Conference on Signals, Systems & Computers, 2003. IEEE, 2003, 1: 87-91.
- [26] 张秀秀, 张立峰. PDF 文件文本内容提取研究.图书情报导刊, 2008, 18(36):118-120.
- [27] 陈云榕, 刘立柱, 叶晗. PDF 文档中 JPEG 图像的自动提取技术研究. 信息工程大学学报,2007, 8(2): 213-216.
- [28] Jordan A, Gauthier F, Hassanshahi B, et al. Unacceptable Behavior: Robust PDF Mal-ware Detection Using Abstract Interpretation. In: Proceedings of the 14th ACM SIGSAC Workshop on Programming Languages and Analysis for Security. 2019: 19-30.
- [29] Li M, Zhou Y, Yu M, et al. Combining static and dynamic analysis for the detection of malicious JavaScript-bearing PDF documents. In: Proceedings of the Computer Science, Technology and Application: Proceedings of the 2016 International Conference on Computer Science, Technology and Application (CSTA2016). 2017: 475-482.
- [30] Bast H, Korzen C. A benchmark and evaluation for text extraction from PDF. In: Proceedings of the ACM/IEEE Joint Conference on Digital Libraries (JCDL). IEEE, 2017: 1-10.

- [31] Python Software Foundation. PDFMiner3K. <https://pypi.org/project/pdfminer3k/>, 2011-05-15.
- [32] The Apache Software Foundation. PDFBox. <http://pdfbox.apache.org/>.
- [33] 牛永洁, 薛苏琴. 基于 PDFBox 抽取学术论文信息的实现. 计算机技术与发展, 2014(12): 61-63.
- [34] NuGet Gallery. iTextSharp. <https://www.nuget.org/packages/iTextSharp/>.
- [35] SOURCEFORGE. PoDoFo. <http://podofo.sourceforge.net/>
- [36] Python Software Foundation. Pdftotext. <https://pypi.org/project/pdftotext/>
- [37] pdftohtml.net. Pdftohtml. <https://www.pdftohtml.net/>
- [38] Crossref. PDF-EXTRACT. <https://www.crossref.org/blog/pdf-extract/>
- [39] Hadjar K, Rigamonti M, Lalanne D, et al. Xed: a new tool for extracting hidden structures from electronic documents. In: Proceedings of the First International Workshop on Document Image Analysis for Libraries, 2004. Proceedings. IEEE, 2004: 212-224.
- [40] what-when-how. Understanding the Carousel Object System (iText 5). <http://what-when-how.com/itext-5/understanding-the-carousel-object-system-itext-5/>.
- [41] Marinai S. Metadata extraction from PDF papers for digital library ingest. In: Proceedings of the 10th International conference on document analysis and recognition. IEEE, 2009: 251-255.
- [42] 朱玲玉. PDF 文档解析与内容脱敏技术研究: [西南交通大学硕士学位论文]. 成都: 西南交通大学, 2018.
- [43] 王津涛, 康晓东, 李玫等. PDF 文件中可识别图像的提取. 计算机工程与设计, 2006 (09): 1539-1541.
- [44] 刘石磊. 面向公式识别的 PDF 图像预处理研究: [河北大学硕士学位论文]. 保定: 河北大学, 2012.
- [45] 唐皓瑾. 一种面向 PDF 文件的表格数据抽取方法的研究与实现: [北京邮电大学硕士学位论文]. 北京: 北京邮电大学, 2015.
- [46] Leonard Rosenthol. Developing with PDF. Sebastopol, California: O'Reilly, 2013.
- [47] 刘琛. PDF 图像数学公式定位算法改进研究: [河北大学硕士学位论文]. 保定: 河北大学, 2016.

- [48] Adobe Developer Support. The Compact Font Format Specification. <https://typekit.files.wordpress.com/2010/12/5176.cff.pdf>, 2003-12-4.
- [49] Adobe Systems Incorporated. Adobe Type 1 Font Format. https://www.adobe.com/content/dam/acom/en/devnet/font/pdfs/T1_SPEC.pdf, 1993.
- [50] Lin X, Gao L, Tang Z, et al. Mathematical formula identification and performance evaluation in PDF documents. *International Journal on Document Analysis and Recognition (IJDAR)*, 2014, 17(3): 239-255.
- [51] 徐宏浩. 仿射不变性图像特征提取与配准算法及应用: [广西师范大学硕士学位论文]. 桂林: 广西师范大学, 2019.
- [52] Hassan T. Object-level document analysis of PDF files. In: *Proceedings of the 9th ACM symposium on Document engineering*. 2009: 47-55.
- [53] 窦方坤, 曹皓伟, 徐建良. 基于文本元素的 PDF 表格区域识别方法研究. *软件导刊*, 2020, 19(01): 113-116.

附录 A 攻读硕士学位期间所发表的学术论文

- [1] MCAF: Developing an Annotationbased Offloading Framework for Mobile Cloud Computing, In:Scientific Programming on 30th April 2020, 第一作者

附录 B 攻读硕士学位期间参加科研项目及申请的专利

（一）参与的科研项目

- [1] 参与PDF学术论文解析与重排版系统的设计与实现

（二）参与申请的专利

- [1] 发明专利：基于移动设备的PDF学术论文重排版系统及方法.已进入实审，申请号：201811421742.6，第三作者

致 谢

岁月匆匆，三年研究生生涯即将画上句号，此时此刻，感慨颇多，在此谨向所有陪伴我成长、带给我鼓励与支持的人表示真诚的谢意！

最需要感谢的是我的导师何黎刚教授，从大四保研并确定导师起，何老师便帮助我一起寻找自己感兴趣的研究方向，在我遇到瓶颈时给予鼓励和支持，在每周一次的例会上，何老师都会和我们一起讨论和分析项目中遇到的难题，指引着我们去发现问题和解决问题，论文的各个环节中何老师都非常有耐心地指导并提出建议和解决方案。何老师对待科学研究态度很严谨，并且生活中也非常自律，我在今后的工作与生活中也会向何老师学习。

我还要感谢我的同门师兄、师弟、师妹们，常诚师兄是我们的大师兄，他是一个非常优秀的人，给予了我非常大的帮助，是我的偶像。我要感谢王斌师兄、夏慧师兄，金鹏、雷良堂师弟、张小瑞师妹，我们一起团队合作完成项目，在业余时间也会一起探讨，在遇到难题时一起集思广益同时在编程能力方面也获得了一定的提高，并且这种团队协作能力也将影响着我以后的工作生涯。

感谢我的家人对我的支持与关怀，是你们陪伴我成长、给我温暖，让我坚定地在求学路上前进。

最后，感谢我的母校，在这里我度过了充满温暖的三年，我将铭记母校对我的栽培！

周忆莲

2020年5月29日